

Projet de Fin de Module : Fondamentaux de Big Data

Sujet 4

Système d'Analyse Prédicative du Parcours Client

et Moteur de Recommandation Dynamique en Temps Réel

Stack : Kafka · Spark Streaming · MongoDB · Cassandra ·
Streamlit · ML

Sources : FakeStore API · DummyJSON API

Deadline : 21 Mai 2026

Réalisé par :

Nom	Rôle
LOUBNA SOUALI	Ingestion en Temps Réel, Production Streaming et Orchestration des Flux (Apache Kafka)
RIME KHAZRAOUI	Conception de l'Architecture Globale, Éthique & Conformité (RGPD) et Ingénierie Documentaire
KHAOULA ANJROUM	Stockage NoSQL, Modélisation IA & Analytics
KHADIJA EL OUARAD	Traitement Distribué en Temps Réel, Calcul Distribué et Streaming (Apache Spark)
RANIA HABIBI	Data Quality Framework, Dashboard et Mode d'Installation Hybride (Mongo Express)

Encadré par : **Pr. Imad Sassi**

Année universitaire : 2025–2026

Table des matières

1	Introduction Générale	6
1.1	Contexte et Problématique	6
1.2	Objectifs du Projet	6
1.3	Jeux de Données Utilisés	7
1.3.1	Clickstream Data	7
1.3.2	Transactions CSV	7
1.3.3	Données Consommateurs	7
2	Architecture et Environnement Technique	8
2.1	Stack Technique	8
2.1.1	Apache Kafka & Streaming	8
2.1.2	Apache Spark Structured Streaming	9
2.1.3	MongoDB (NoSQL Documents)	10
2.1.4	Cassandra (NoSQL Distribué)	10
2.1.5	Python & Bibliothèques ML	11
2.2	Organisation du Projet	11
2.2.1	Architecture Globale du Pipeline	11
2.2.2	Environnement Virtuel (.venv)	12
3	Profiling Producteurs et Consommateurs	14
	Partie 1 : Le Producteur	14
3.1	Rôle du Producteur dans l'Architecture	14
3.2	Sources de Données Externes	14
3.2.1	API FakeStore (produits e-commerce)	14
3.2.2	API DummyJSON (avis et ratings)	15
3.3	Génération des Événements	15
3.3.1	Événements Clickstream (navigation utilisateur)	15
3.3.2	Événements Transactions (paiements)	15
3.3.3	Événements Service Client (support)	15
3.4	Logique de Publication vers Kafka	16

3.4.1	Connexion au Broker Kafka	16
3.4.2	Sérialisation JSON des Messages	16
3.4.3	Règles de Routage vers les 3 Topics	16
3.5	Boucle de Streaming en Continu	16
Partie 2 : Le Consommateur		16
3.6	Rôle du Consommateur dans l'Architecture	16
3.7	Souscription aux Topics Kafka	17
3.7.1	Lecture du Topic clickstream	17
3.7.2	Lecture du Topic transactions	17
3.7.3	Lecture du Topic service_client	17
3.8	Traitement et Transformation des Messages	17
3.9	Persistance des Données Consommées	18
3.9.1	Stockage dans MongoDB	18
3.9.2	Stockage dans Cassandra	18
3.10	Gestion des Erreurs et Robustesse	18
4	Traitement Distribué avec Apache Spark	19
4.1	Pipeline Spark Structured Streaming	19
4.2	Nettoyage et Prétraitement des Données	20
4.3	Calcul du Score d'Engagement	20
4.4	Enrichissement des Données en Temps Réel	20
4.5	Gestion des Checkpoints et Tolérance aux Pannes	21
5	Analyse NLP et Profiling Comportemental	22
5.1	Pipeline NLP Complet	22
5.1.1	Tokenisation avec NLTK	22
5.1.2	Analyse de Sentiment (TextBlob)	22
5.1.3	Analyse de Subjectivité	23
5.1.4	Comptage Mots Positifs/Négatifs	23
5.2	Classification du Profil Comportemental	23
5.2.1	Les 5 Profils Utilisateurs	23
5.2.2	Logique de Détection Automatique	23
5.3	Résultats NLP Obtenus	24
5.4	Intégration NLP dans le Pipeline Spark	25
6	Stockage NoSQL	26
6.1	MongoDB : Stockage des Profils et Historiques	26
6.1.1	Collections et schémas de documents	26
6.1.2	Opérations CRUD et indexation	26
6.2	Cassandra : Stockage des Événements Temps Réel	27

6.2.1	Schéma CQL et stratégie de partitionnement	27
6.2.2	Écriture des flux Kafka vers Cassandra	27
6.3	Comparaison MongoDB vs Cassandra	27
7	Modélisation IA & Analytics	29
7.1	Préparation des Données	29
7.1.1	Feature Engineering (28 features)	29
7.1.2	Encodage des variables catégorielles (LabelEncoder)	29
7.1.3	Équilibrage des classes avec SMOTE	30
7.1.4	Split Train/Test 80%/20%	30
7.2	Entraînement des 3 Modèles	30
7.2.1	Modèle 1 : Régression Logistique	30
7.2.2	Modèle 2 : Random Forest	30
7.2.3	Modèle 3 : Gradient Boosting (Scikit-learn)	31
7.3	Étude Comparative des Modèles	31
7.3.1	Métriques d'évaluation	31
7.3.2	Temps d'entraînement et d'inférence	31
7.3.3	Tableau récapitulatif et choix du meilleur modèle	31
7.4	Prédiction et Recommandations	32
7.4.1	Déploiement du modèle (joblib)	32
7.4.2	Logique de recommandation intelligente	32
7.5	Sauvegarde des Modèles	32
8	Data Quality Framework	33
8.1	Analyse de Complétude	33
8.2	Détection des Doublons	34
8.3	Vérification de Cohérence des Valeurs	34
8.4	Statistiques Descriptives	35
8.5	Analyse des Biais Algorithmiques	36
8.6	Visualisations Data Quality	37
9	Visualisation & Reporting (Dashboard)	39
9.1	Architecture du Dashboard Streamlit	39
9.2	KPIs en Temps Réel	40
9.3	Tunnel de Conversion et Graphiques	41
9.4	Nuage de Mots Dynamique (Reviews)	43
9.5	Système d'Alertes Automatiques	44
9.6	Auto-Refresh et Connexion MongoDB Live	45
10	Interface d'Administration MongoDB : Mongo Express	46

10.1	Présentation Générale	46
10.2	Configuration et Déploiement via Docker Compose	46
10.2.1	Configuration avec Replica Set (architecture haute disponibilité)	47
10.2.2	Configuration simple (architecture standalone)	47
10.2.3	Description des Paramètres	47
10.3	Accès à l'Interface	47
10.4	Exploration de l'Interface : Exemple Pas à Pas	48
10.4.1	Étape 1 : Page d'accueil (Liste des bases de données)	48
10.4.2	Étape 2 : Statut du serveur MongoDB	48
10.4.3	Étape 3 : Exploration de la base admin	49
10.4.4	Étape 4 : Inspection de la collection system.version	49
10.4.5	Étape 5 : Outils de gestion des collections	51
10.5	Bilan	51
11	Modes d'Installation Hybride	53
11.1	Mode 1 : Installation Locale (Machine Physique)	53
11.1.1	Caractéristiques clés	53
11.2	Mode 2 : Distribué (Docker-Compose)	53
11.2.1	Caractéristiques clés	54
11.3	Comparaison des Performances entre les 2 Modes	54
12	Éthique & Conformité RGPD	55
12.1	Protection des Données Personnelles	55
12.1.1	Application du Principe de Minimisation des Données	55
12.1.2	Implémentation du Droit à l'Oubli (Article 17)	56
12.2	Biais Algorithmiques et Équité	56
12.2.1	Détection des Biais dans nos Données	56
12.2.2	Stratégies Techniques d'Atténuation	56
12.2.3	Résultats Obtenus	57
12.3	Pseudonymisation des Identifiants Utilisateurs	57
12.3.1	Mécanisme de Hachage Salé Déterministe	57
12.3.2	Chiffrement des Données Textuelles Sensibles	58
12.4	Limitations et Responsabilité de l'IA	58
12.4.1	Analyse Critique du F1-Score et Impact Métier	58
12.4.2	Le Principe de Supervision Humaine	58
12.4.3	Limites du Projet	59
13	Conclusion et Perspectives	60
13.1	Bilan Technique du Projet	60
13.2	Limites et Difficultés Rencontrées	61

13.3 Perspectives et Améliorations	61
13.4 Apports Personnels et Compétences Acquises	62
Références	64

Chapitre 1

Introduction Générale

1.1 Contexte et Problématique

Dans un monde de plus en plus connecté, les entreprises du secteur e-commerce génèrent chaque jour des volumes massifs de données hétérogènes : comportements de navigation, transactions d'achat, interactions avec le support client, et profils consommateurs. Ces données, si elles sont bien exploitées, constituent une source précieuse d'informations stratégiques permettant d'améliorer l'expérience client, d'optimiser les ventes et d'anticiper les tendances du marché.

Cependant, la gestion et l'analyse de ces données posent plusieurs défis majeurs. D'abord, leur volume considérable nécessite des infrastructures de stockage distribuées et performantes. Ensuite, leur hétérogénéité (données structurées, semi-structurées et non structurées) exige des outils capables de les traiter de manière unifiée. Enfin, la vélocité avec laquelle ces données sont produites impose des pipelines de traitement en temps réel ou quasi-réel.

C'est dans ce contexte que s'inscrit notre projet Big Data, qui vise à concevoir une solution complète d'ingestion, de stockage, d'analyse et de visualisation de données e-commerce, en mobilisant les technologies modernes du Big Data.

1.2 Objectifs du Projet

Ce projet a pour objectif principal de mettre en place un pipeline Big Data end-to-end capable de traiter des données e-commerce issues de sources multiples. Plus précisément, les objectifs sont les suivants :

- Ingérer et enrichir les données brutes de clickstream, de transactions et de support client afin de les rendre exploitables pour l'analyse.

- Évaluer la qualité des données collectées en détectant les valeurs manquantes, les doublons et les incohérences.
- Stocker efficacement les données dans deux bases NoSQL complémentaires (MongoDB et Cassandra) et comparer leurs performances.
- Appliquer des algorithmes de Machine Learning pour modéliser les comportements des consommateurs et prédire leurs actions futures.
- Visualiser les résultats à travers un tableau de bord interactif présentant les indicateurs clés de performance (KPI) du système.

1.3 Jeux de Données Utilisés

Pour atteindre ces objectifs, trois jeux de données complémentaires ont été utilisés tout au long du projet :

1.3.1 Clickstream Data

Le jeu de données clickstream (`clickstream.csv`) enregistre les traces de navigation des utilisateurs sur une plateforme e-commerce. Chaque ligne représente un événement de navigation et contient des informations telles que : l'identifiant de la session, la page visitée, la catégorie de produit consultée, la source de trafic (organique, publicité, réseaux sociaux), le profil de l'utilisateur, ainsi que le sentiment associé à la visite.

Ce dataset constitue la base principale du projet. Il a été enrichi, nettoyé et transformé pour produire le fichier `final_clickstream.csv`, utilisé ensuite pour l'entraînement des modèles de Machine Learning et l'alimentation du tableau de bord.

1.3.2 Transactions CSV

Le fichier `final_transactions.csv` contient l'historique des achats effectués par les utilisateurs. Il inclut des informations sur les montants des commandes, les produits achetés, les dates de transaction et les identifiants clients. Ce jeu de données permet d'analyser les comportements d'achat, de calculer des indicateurs comme la valeur moyenne des commandes (AOV) et d'identifier les segments de clients les plus rentables.

1.3.3 Données Consommateurs

Le fichier `final_support.csv` regroupe les interactions des clients avec le service support. Il contient les tickets ouverts, les types de problèmes signalés et les résolutions apportées. Couplé aux données clickstream et transactions, il permet d'établir une vision à 360° du parcours client, d'identifier les points de friction dans l'expérience utilisateur et d'améliorer la qualité du service proposé.

Chapitre 2

Architecture et Environnement Technique

2.1 Stack Technique

Notre projet repose sur une stack Big Data moderne combinant plusieurs technologies complémentaires, orchestrées via Docker. L'ensemble de l'infrastructure est défini dans le fichier `docker-compose.yml` et les dépendances Python sont gérées dans l'environnement virtuel `.venv`.

2.1.1 Apache Kafka & Streaming

Apache Kafka est le système de messagerie distribué au cœur de notre architecture. Il joue le rôle de bus de données en temps réel entre le producteur et les consommateurs.

Dans notre projet, Kafka est déployé via Docker en mode KRaft (sans Zookeeper) avec la configuration suivante dans `docker-compose.yml` :

```
kafka:
  image: confluentinc/cp-kafka:7.4.0
  ports:
    - "9092:9092"
  environment:
    KAFKA_NODE_ID: 1
    KAFKA_PROCESS_ROLES: broker,controller
    KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,CONTROLLER
                      ://0.0.0.0:9093
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
    KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    CLUSTER_ID: Mku30EVBNTcwNTJENDM2Qk
```

Trois topics Kafka sont créés pour séparer les flux de données :

Topic	Rôle	Données
clickstream	Navigation utilisateurs	session_id, action, produit, source
transactions	Paielements et achats	montant, méthode, statut
service_client	Support et réclamations	motif, canal, résolu

TABLE 2.1 – Topics Kafka du projet

Le producteur `producteur.py` connecte Kafka via `kafka-python` et envoie les données en JSON sérialisé :

```
producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)
```

Les données sont générées en temps réel depuis deux APIs externes (FakeStore API pour les produits et DummyJSON pour les reviews) avec un intervalle de 0.5 seconde entre chaque événement.

2.1.2 Apache Spark Structured Streaming

Apache Spark est utilisé pour le traitement distribué des flux de données en temps réel. Notre projet utilise **Spark Structured Streaming** avec le connecteur Kafka :

```
org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.0
```

Spark est installé localement dans la VM (`/opt/spark`) et configuré en mode `local[2]` pour utiliser 2 cœurs. Il traite les micro-batches toutes les 10 secondes depuis les trois topics Kafka.

Les fichiers Spark du projet sont :

Fichier	Rôle
<code>consumer_mongodb.py</code>	Sauvegarde streaming → MongoDB
<code>consumer_cassandra.py</code>	Sauvegarde streaming → Cassandra
<code>consomateur.py</code>	Consumer IA avec NLP et ML
<code>enrichir_mongodb.py</code>	Enrichissement des données
<code>data_quality.py</code>	Contrôle qualité des données

TABLE 2.2 – Scripts Spark du projet

2.1.3 MongoDB (NoSQL Documents)

MongoDB est la base de données NoSQL principale du projet pour le stockage des données enrichies. Elle est déployée via Docker :

```
mongodb:
  image: mongo:latest
  ports:
    - "27017:27017"
```

Trois collections sont utilisées dans la base `ecommerce` :

Collection	Contenu	Taille approximative
clickstream	Événements de navigation enrichis	~280 KB
transactions	Paiements et achats	~89 KB
service_client	Tickets de support	~27 KB

TABLE 2.3 – Collections MongoDB

La connexion Python se fait via `pymongo` :

```
from pymongo import MongoClient
client = MongoClient("localhost", 27017)
db = client["ecommerce"]
```

MongoDB a été choisi pour sa flexibilité de schéma, ses agrégations puissantes, et sa compatibilité native avec les données JSON produites par Kafka.

2.1.4 Cassandra (NoSQL Distribué)

Apache Cassandra est utilisé en complément de MongoDB pour les tests de performance et la comparaison NoSQL. Le fichier `comparaison_nosql.py` effectue des benchmarks d'insertion et de lecture entre les deux bases.

La connexion Python se fait via `cassandra-driver` :

```
from cassandra.cluster import Cluster
cluster = Cluster(['localhost'], port=9042)
session = cluster.connect('ecommerce')
```

Le keyspace `ecommerce` contient les mêmes trois tables que MongoDB : `clickstream`, `transactions`, et `service_client`, avec des clés primaires composites (`session_id`, `timestamp`) adaptées au modèle colonne de Cassandra.

2.1.5 Python & Bibliothèques ML

L'ensemble du projet est développé en **Python 3.12**. Les bibliothèques utilisées sont regroupées par catégorie :

Streaming & Kafka :

Bibliothèque	Version	Rôle
kafka-python	latest	Producteur et consommateur Kafka
pyspark	3.3.0+	Spark Structured Streaming

Bases de données :

Bibliothèque	Version	Rôle
pymongo	4.17.0	Connexion MongoDB
cassandra-driver	latest	Connexion Cassandra

Machine Learning :

Bibliothèque	Version	Rôle
scikit-learn	latest	Random Forest, Logistic Regression, GBT
imbalanced-learn	latest	SMOTE pour rééquilibrage des classes
joblib	latest	Sauvegarde et chargement des modèles .pkl
pandas	latest	Manipulation des DataFrames
numpy	latest	Calculs numériques

NLP :

Visualisation :

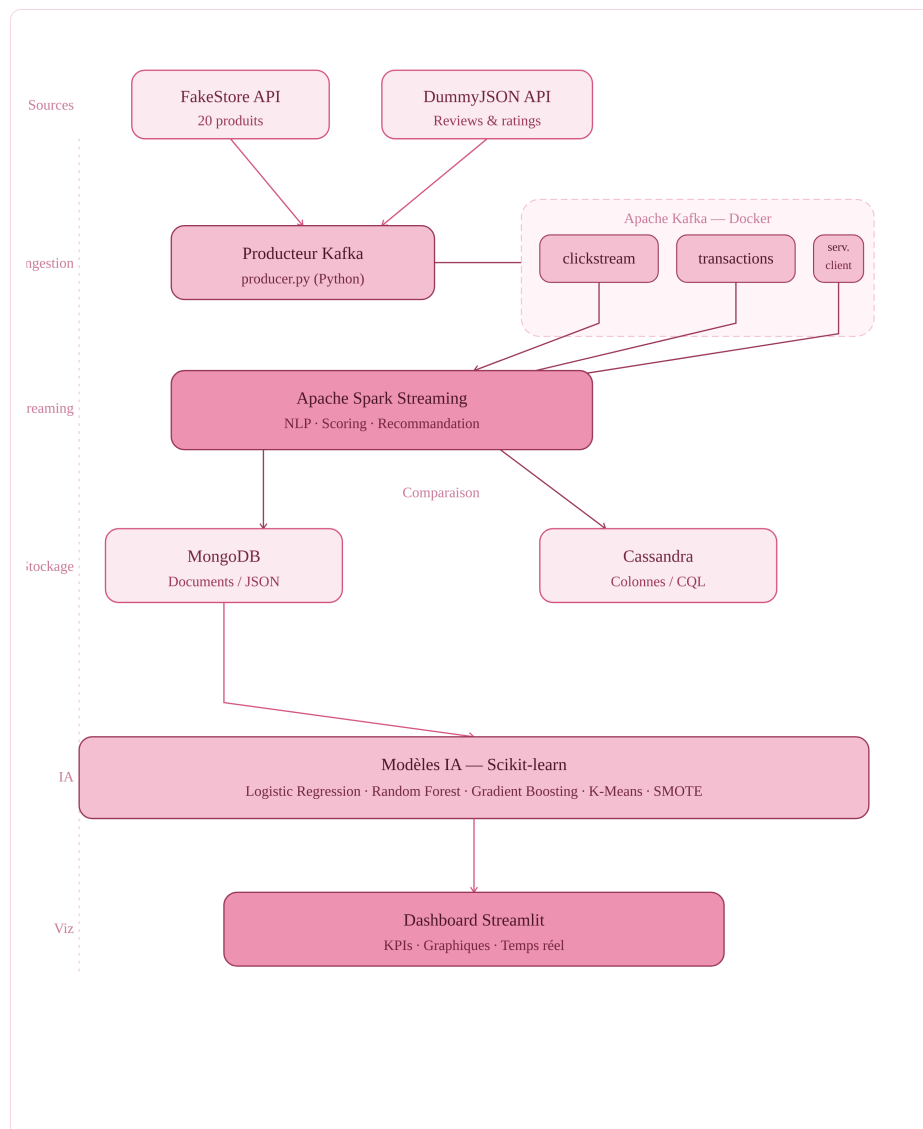
2.2 Organisation du Projet

2.2.1 Architecture Globale du Pipeline

La figure ci-dessous illustre l'architecture complète du pipeline de traitement des données e-commerce, organisée en cinq couches distinctes :

Bibliothèque	Version	Rôle
<code>textblob</code>	latest	Analyse de sentiment et subjectivité
<code>nltk</code>	latest	Tokenisation, stopwords, lemmatisation

Bibliothèque	Version	Rôle
<code>dash</code>	latest	Dashboard interactif temps réel
<code>plotly</code>	latest	Graphiques interactifs

FIGURE 2.1 – `pipeline_ecommerce`

2.2.2 Environnement Virtuel (.venv)

L'environnement virtuel Python est créé dans le dossier `.venv/` avec **Python 3.12**. Il isole toutes les dépendances du projet du système d'exploitation.

Création et activation :

```
# Creation
python3 -m venv .venv

# Activation (Linux/Mac)
source .venv/bin/activate

# Verification
python --version # Python 3.12.x
```

L'environnement virtuel contient les packages installés via pip dans `.venv/lib/python3.12/site-packages` dont les principaux sont `kafka-python`, `pymongo`, `textblob`, `nltk`, `scikit-learn`, `imbalanced-learn`, `dash`, et `plotly`.

Avantages de l'environnement virtuel :

- Isolation complète des dépendances du projet
- Reproductibilité de l'environnement sur d'autres machines
- Évite les conflits de versions entre projets
- Facilite le déploiement et le partage du projet

Chapitre 3

Profiling Producteurs et Consommateurs

Partie 1 : Le Producteur

Fichier principal : `producer.py`

3.1 Rôle du Producteur dans l'Architecture

Le producteur constitue le point d'entrée du pipeline de données. Son rôle est de simuler le comportement réel d'utilisateurs sur une plateforme e-commerce en générant des événements en continu et en les publiant vers Apache Kafka. Il joue le rôle de source de données temps réel, alimentant trois topics Kafka distincts : `clickstream`, `transactions` et `service_client`.

3.2 Sources de Données Externes

Le producteur s'appuie sur deux APIs publiques pour obtenir des données réalistes sur les produits et les avis clients.

3.2.1 API FakeStore (produits e-commerce)

L'API FakeStore (<https://fakestoreapi.com/products>) fournit un catalogue de 20 produits réels avec leurs attributs : identifiant, titre, catégorie, prix et note. Ces produits servent de base pour générer les événements de navigation et d'achat simulés.

3.2.2 API DummyJSON (avis et ratings)

L'API DummyJSON (<https://dummyjson.com/products?limit=30&select=title,reviews,rating>) fournit 30 produits supplémentaires accompagnés de leurs avis clients et leurs notes. Les commentaires extraits de cette API sont utilisés pour enrichir les événements clickstream avec des données textuelles réalistes, qui seront ensuite traitées par le module NLP.

3.3 Génération des Événements

Le producteur génère trois types d'événements distincts pour simuler les différentes interactions d'un utilisateur sur la plateforme.

3.3.1 Événements Clickstream (navigation utilisateur)

Chaque événement clickstream représente une interaction d'un utilisateur avec un produit. Il contient les champs suivants :

- `user_id` : identifiant aléatoire entre 1 et 200
- `product_id` / `product_title` / `category` / `price` / `rating` : informations du produit
- `action` : type d'interaction choisi aléatoirement parmi `view`, `add_to_cart`, `purchase`, `abandon`, `search`
- `session_id` : identifiant unique de session
- `page_visitee` / `duree_visite` / `source` : données de navigation simulées
- `review_text` : avis client tiré aléatoirement depuis DummyJSON
- `timestamp` : horodatage Unix de l'événement

3.3.2 Événements Transactions (paiements)

Les événements de transaction sont générés uniquement pour les actions d'achat. Ils contiennent en plus les champs financiers :

- `montant_paye` : prix du produit avec légère variation aléatoire
- `methode_paiement` : choisi parmi `carte`, `paypal`, `virement`
- `statut_paiement` : `success`, `pending` ou `failed`

3.3.3 Événements Service Client (support)

Ces événements simulent les interactions avec le service client :

- `motif_contact` : raison du contact (`remboursement`, `livraison`, `produit_defectueux`, etc.)
- `canal` : moyen de contact (`email`, `chat`, `telephone`)

- `resolu` : booléen indiquant si le problème a été résolu

3.4 Logique de Publication vers Kafka

3.4.1 Connexion au Broker Kafka

La connexion au broker Kafka se fait via la bibliothèque `kafka-python` en pointant vers `localhost:9092`, port exposé par le container Docker Kafka.

3.4.2 Sérialisation JSON des Messages

Chaque événement Python (dictionnaire) est sérialisé en JSON puis encodé en UTF-8 avant publication, grâce au `value_serializer` configuré dans le `KafkaProducer`.

3.4.3 Règles de Routage vers les 3 Topics

Le producteur applique une logique de routage selon le type d'événement :

- Navigation utilisateur → Topic `clickstream`
- Paiement / achat → Topic `transactions`
- Contact support → Topic `service_client`

3.5 Boucle de Streaming en Continu

Le producteur tourne en boucle infinie (`while True`) : pour chaque produit du catalogue FakeStore, il génère un événement, lui associe un avis aléatoire depuis DummyJSON, et publie le message vers Kafka. Un délai de 0.5 seconde entre chaque envoi simule un flux réaliste d'événements utilisateurs.

Partie 2 : Le Consommateur

Fichiers concernés : `consumer.py`, `consumer_mongodb.py`, `consumer_cassandra.py`

3.6 Rôle du Consommateur dans l'Architecture

Le consommateur constitue la couche de traitement et de persistance du pipeline. Implémenté avec Apache Spark Structured Streaming, il souscrit aux trois topics Kafka, parse et enrichit les données en temps réel, puis les stocke dans les bases NoSQL.

Il joue le rôle de pont entre la couche d'ingestion (Kafka) et la couche de stockage (MongoDB/Cassandra).

3.7 Souscription aux Topics Kafka

3.7.1 Lecture du Topic clickstream

Spark lit en continu les messages du topic `clickstream` depuis le broker `localhost:9092`.

Le schéma de parsing est défini avec `StructType` contenant 13 champs : `session_id`, `user_id`, `product_id`, `product_title`, `category`, `price`, `rating`, `action`, `page_visitee`, `du-ree_visite`, `source`, `review_text`, `timestamp`.

3.7.2 Lecture du Topic transactions

Le topic `transactions` est lu avec un schéma de 9 champs incluant les informations financières : `montant_paye`, `methode_paiement`, `statut_paiement`.

3.7.3 Lecture du Topic service_client

Le topic `service_client` est lu avec un schéma de 6 champs : `session_id`, `user_id`, `motif_contact`, `canal`, `resolu`, `timestamp`.

3.8 Traitement et Transformation des Messages

Après parsing JSON, le consommateur applique plusieurs transformations sur le flux `clickstream` :

- **Nettoyage NLP** : la colonne `review_clean` est générée en mettant le texte en minuscules et en supprimant tous les caractères non alphabétiques via `regexp_replace`.
- **Scoring des actions** : une colonne `action_score` est calculée selon l'intention d'achat : `purchase` vaut 5, `add_to_cart` vaut 3, `view` et `search` valent 1, `abandon` vaut -1.
- **Recommandation temps réel** : une colonne `recommandation` est générée automatiquement selon l'action : coupon -20% pour les abandons, livraison gratuite pour les ajouts au panier, suggestions de produits similaires pour les achats.

3.9 Persistance des Données Consommées

3.9.1 Stockage dans MongoDB

Le `consumer_mongodb.py` utilise `foreachBatch` pour écrire chaque micro-batch dans MongoDB. Trois collections sont alimentées dans la base `ecommerce` : `clickstream`, `transactions` et `service_client`. La connexion MongoDB se fait via `pymongo` sur `localhost:27017`.

3.9.2 Stockage dans Cassandra

Le `consumer_cassandra.py` utilise la même logique `foreachBatch` mais persiste les données dans Cassandra via le driver `cassandra-driver`. Le keyspace `bigdata_projet` contient trois tables créées préalablement avec des clés primaires composites (`session_id`, `timestamp`) pour optimiser les requêtes temporelles.

3.10 Gestion des Erreurs et Robustesse

Plusieurs mécanismes de robustesse ont été mis en place :

- `failOnDataLoss = false` : Spark ne s'arrête pas si des messages Kafka sont perdus.
- `startingOffsets = latest` : le consommateur ne relit pas l'historique au redémarrage.
- Vérification `if df.count() == 0` avant toute écriture NoSQL pour éviter les insertions vides.
- Points de contrôle (`checkpointLocation`) pour permettre la reprise après panne.
- Délais de connexion (`connect_timeout=30`, `default_timeout=60`) pour Cassandra afin de gérer les lenteurs de démarrage du container.

Chapitre 4

Traitement Distribué avec Apache Spark

4.1 Pipeline Spark Structured Streaming

Apache Spark Structured Streaming constitue le cœur du traitement en temps réel de notre architecture. Il permet de consommer les données depuis les topics Kafka et de les traiter de manière continue et distribuée.

La session Spark est configurée comme suit :

```
spark = SparkSession.builder \
    .appName("Clickstream_Consumer_IA") \
    .master("local[2]") \
    .config("spark.jars.packages",
            "org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.0") \
    .config("spark.sql.shuffle.partitions", "2") \
    .config("spark.executor.memory", "1g") \
    .config("spark.driver.memory", "1g") \
    .config("spark.memory.fraction", "0.6") \
    .config("spark.sql.streaming.forceDeleteTempCheckpointLocation", "
            true") \
    .getOrCreate()
```

Notre pipeline traite trois topics Kafka en parallèle :

- **clickstream** : événements de navigation des utilisateurs
- **transactions** : paiements et achats effectués
- **service_client** : tickets de support et réclamations

Chaque topic est lu via la fonction `read_kafka()` définie dans `consumer.py` qui configure les paramètres de connexion, le mode de lecture (`latest`), et la tolérance aux pertes de données (`failOnDataLoss: false`).

4.2 Nettoyage et Prétraitement des Données

Les données brutes reçues depuis Kafka sont au format JSON sérialisé. Le prétraitement commence par la désérialisation via `from_json()` en utilisant des schémas typés définis explicitement dans `consumer.py`.

Trois schémas sont définis :

Schéma Clickstream comprend les champs : `session_id`, `user_id`, `product_id`, `product_title`, `category`, `price`, `rating`, `action`, `page_visitee`, `duree_visite`, `source`, `review_text`, et `timestamp`.

Schéma Transactions comprend : `session_id`, `user_id`, `product_id`, `product_title`, `category`, `montant_paye`, `methode_paieement`, `statut_paieement`, et `timestamp`.

Schéma Service Client comprend : `session_id`, `user_id`, `motif_contact`, `canal`, `resolu`, et `timestamp`.

Le nettoyage du texte des reviews est effectué par suppression des caractères spéciaux et conversion en minuscules :

```
.withColumn("review_clean",
            lower(regexp_replace(col("review_text"), "[^a-zA-Z\\s]", "")))
```

4.3 Calcul du Score d'Engagement

Un score d'engagement est calculé dans `consumer.py` pour chaque événement clickstream afin de quantifier l'intention d'achat de l'utilisateur :

```
.withColumn("action_score",
            when(col("action") == "purchase",      5)
            .when(col("action") == "add_to_cart", 3)
            .when(col("action") == "view",        1)
            .when(col("action") == "search",      1)
            .when(col("action") == "abandon",     -1)
            .otherwise(0))
```

4.4 Enrichissement des Données en Temps Réel

Les données sont enrichies en temps réel dans `consumer.py` par l'ajout d'une colonne `recommandation` générée automatiquement selon l'action de l'utilisateur :

```
.withColumn("recommandation",
            when(col("action") == "abandon",      "Envoyer coupon -20%"))
```

Action	Score	Interprétation
purchase	+5	Achat confirmé (engagement maximal)
add_to_cart	+3	Intention d'achat forte
view	+1	Intérêt pour le produit
search	+1	Recherche active
abandon	-1	Signal négatif (risque de perte)

TABLE 4.1 – Score d'engagement par action

```
.when(col("action") == "add_to_cart", "Proposer livraison gratuite")
.when(col("action") == "purchase", "Suggerer produits similaires")
.otherwise("Continuer navigation"))
```

4.5 Gestion des Checkpoints et Tolérance aux Pannes

Spark Structured Streaming garantit la tolérance aux pannes grâce au mécanisme de checkpointing défini dans `consumer.py`. Chaque stream possède son propre répertoire :

```
.option("checkpointLocation", "/tmp/checkpoint_consumer_clicks")
.option("checkpointLocation", "/tmp/checkpoint_consumer_tx")
.option("checkpointLocation", "/tmp/checkpoint_consumer_support")
```

Les micro-batches sont déclenchés toutes les 10 secondes. En cas de panne, Spark reprend automatiquement depuis le dernier checkpoint sans perte de données.

Chapitre 5

Analyse NLP et Profiling Comportemental

5.1 Pipeline NLP Complet

L'analyse NLP est intégrée dans `consumer.py` pour analyser les reviews des utilisateurs et extraire des informations comportementales en temps réel.

5.1.1 Tokenisation avec NLTK

La tokenisation décompose le texte en tokens individuels, supprime les stopwords, et applique la lemmatisation :

```
def nb_tokens_udf(text):
    try:
        tokens = word_tokenize(str(text).lower())
        tokens = [t for t in tokens if t.isalpha()
                  and t not in stop_words]
        return len(tokens)
    except:
        return 0
```

Les ressources NLTK utilisées dans `consumer.py` sont : `punkt`, `stopwords`, `wordnet`, et `punkt_tab`.

5.1.2 Analyse de Sentiment (TextBlob)

TextBlob est utilisé dans `consumer.py` pour calculer la polarité du sentiment :

```
def analyser_sentiment_udf(text):
    try:
```

```

score = TextBlob(str(text)).sentiment.polarity
if score > 0.2: return "positif"
if score < -0.2: return "negatif"
return "neutre"
except:
    return "neutre"

```

Polarité	Classification
> 0.2	positif
Entre -0.2 et 0.2	neutre
< -0.2	négatif

TABLE 5.1 – Classification de la polarité des sentiments

5.1.3 Analyse de Subjectivité

```

def subjectivite_udf(text):
    try:
        return round(TextBlob(str(text)).sentiment.subjectivity, 3)
    except:
        return 0.0

```

5.1.4 Comptage Mots Positifs/Négatifs

```

positive_words = ['great', 'excellent', 'good', 'happy', 'satisfied',
                  'awesome', 'love', 'perfect', 'recommended', '
                  pleased',
                  'impressed', 'wonderful', 'fantastic', 'amazing']

negative_words = ['poor', 'bad', 'disappointed', 'unhappy', 'terrible'
                  ,
                  'worst', 'dissatisfied', 'waste', 'awful', 'horrible'
                  ,
                  'useless', 'broken', 'cheap', 'slow', 'expensive']

score_nlp = nb_positifs - nb_negatifs

```

5.2 Classification du Profil Comportemental

5.2.1 Les 5 Profils Utilisateurs

5.2.2 Logique de Détection Automatique

Profil	Description	Caractéristiques
client_fidele	Client régulier et satisfait	Taux achat > 40%, score > 10
chasseur_promos	Sensible aux promotions	Abandons > achats, sentiment négatif
acheteur_impulsif	Achète rapidement	Achats > 0, actions < 3
comparateur	Compare avant d'acheter	Actions > 10, taux achat < 10%
visiteur_standard	Comportement normal	Aucun critère spécifique

TABLE 5.2 – Typologie des profils utilisateurs

```
def predire_intention(row):
    if taux > 0.4 and score_eng > 10:
        profil = "client_fidele"
    elif abandons > achats and sentiment == 'negatif':
        profil = "chasseur_promos"
    elif achats > 0 and actions < 3:
        profil = "acheteur_impulsif"
    elif actions > 10 and taux < 0.1:
        profil = "comparateur"
    else:
        profil = "visiteur_standard"
```

5.3 Résultats NLP Obtenus

Champ	Type	Exemple
sentiment	String	"positif" / "negatif" / "neutre"
sent_score	Float	0.45
subjectivite	Float	0.62
nb_tokens	Int	8
nb_mots_positifs	Int	3
nb_mots_negatifs	Int	0
score_nlp	Int	+3
profil	String	"client_fidele"

TABLE 5.3 – Champs générés par l'analyse NLP

5.4 Intégration NLP dans le Pipeline Spark

Le NLP est intégré dans `consumer.py` via `foreachBatch()`. Pour chaque micro-batch de 10 secondes, la fonction `process_clicks_batch()` est appelée :

```
def process_clicks_batch(df, epoch_id):
    rows = df.collect()
    for row in rows[:5]:
        row_dict = row.asDict()
        prediction, proba, sentiment, profil = predire_intention(
            row_dict)

        if prediction == 1:
            if proba > 80:
                recommandation = f"ACHAT PROBABLE ({proba}%)"
            else:
                recommandation = f"ACHAT POSSIBLE ({proba}%)"
        else:
            if sentiment == 'negatif':
                recommandation = f"RISQUE ABANDON : Coupon -20%"
            else:
                recommandation = f"NAVIGATION NORMALE"
```

Cette architecture garantit un traitement NLP en temps réel avec une latence inférieure à 10 secondes, permettant des recommandations personnalisées instantanées pour chaque utilisateur.

Chapitre 6

Stockage NoSQL

6.1 MongoDB : Stockage des Profils et Historiques

6.1.1 Collections et schémas de documents

MongoDB est utilisé comme base de données principale pour stocker les événements traités par Spark Streaming. La base de données s'appelle `ecommerce` et contient trois collections distinctes, chacune correspondant à un topic Kafka.

La collection `clickstream` stocke les événements de navigation enrichis avec les colonnes suivantes : `session_id`, `user_id`, `product_id`, `product_title`, `category`, `price`, `rating`, `action`, `page_visitee`, `duree_visite`, `source`, `review_text`, `review_clean`, `action_score`, `recommandation` et `timestamp`. Chaque document est un objet JSON flexible, ce qui permet d'ajouter de nouveaux champs sans modifier le schéma.

La collection `transactions` stocke les informations financières : `session_id`, `user_id`, `product_id`, `product_title`, `category`, `montant_paye`, `methode_paiement`, `statut_paiement` et `timestamp`.

La collection `service_client` stocke les tickets de support : `session_id`, `user_id`, `motif_contact`, `canal`, `resolu` et `timestamp`.

6.1.2 Opérations CRUD et indexation

Les opérations d'écriture dans MongoDB sont réalisées via la méthode `insert_many()` de la bibliothèque `pymongo`, qui permet d'insérer plusieurs documents en une seule opération pour optimiser les performances. La connexion se fait sur `localhost:27017` et est ouverte puis fermée à chaque micro-batch pour éviter les fuites de connexions.

La fonction `foreachBatch` de Spark Structured Streaming est utilisée pour déclencher

l'écriture toutes les 10 secondes. Avant chaque insertion, une vérification `df.count() == 0` est effectuée pour éviter les appels inutiles à MongoDB sur des batches vides.

6.2 Cassandra : Stockage des Événements Temps Réel

6.2.1 Schéma CQL et stratégie de partitionnement

Cassandra utilise le keyspace `bigdata_projet` avec une stratégie de réplication `SimpleStrategy` et un facteur de réplication de 1. Trois tables ont été créées en CQL avec des clés primaires composites (`session_id`, `timestamp`), ce qui permet de partitionner les données par session et de trier chronologiquement les événements à l'intérieur de chaque partition.

La table `clickstream` contient les mêmes champs que MongoDB avec en plus `review_clean`, `action_score` et `recommandation`. La table `transactions` stocke les informations financières, et la table `service_client` stocke les tickets de support avec le champ `resolu` de type `BOOLEAN`.

Ce choix de clé primaire est justifié par le modèle d'accès Cassandra : les requêtes portent généralement sur une session spécifique ou sur une plage temporelle, ce qui correspond parfaitement à la structure (`session_id`, `timestamp`).

6.2.2 Écriture des flux Kafka vers Cassandra

L'écriture dans Cassandra utilise le driver `cassandra-driver` via la classe `Cluster`. Pour chaque micro-batch, une connexion est établie vers l'adresse IP du container Docker (172.19.0.4) sur le port 9042, avec un timeout de connexion de 30 secondes et un timeout de requête de 60 secondes pour gérer les lenteurs de démarrage.

Les insertions sont effectuées ligne par ligne avec des requêtes CQL préparées via `session.execute()`, garantissant la cohérence des types de données. Le temps d'exécution de chaque batch est mesuré et affiché pour permettre le monitoring des performances.

6.3 Comparaison MongoDB vs Cassandra

Le benchmark effectué via `comparaison_nosql.py` sur 1000 insertions donne les résultats suivants :

MongoDB se révèle nettement plus rapide pour ce cas d'usage e-commerce, avec une vitesse d'écriture 5 fois supérieure et des lectures filtrées quasi instantanées grâce à l'indexation automatique. Ce résultat s'explique par la nature des données : des documents

Métrique	MongoDB	Cassandra
Écriture 1000 docs (s)	0.431	2.464
Vitesse écriture (docs/s)	2323	406
Lecture COUNT (s)	0.001	0.099
Query filtrée (s)	0.007	N/A

TABLE 6.1 – Benchmark MongoDB vs Cassandra

JSON flexibles avec des requêtes filtrées par catégorie ou action, ce qui correspond exactement au modèle de données de MongoDB.

Cassandra reste néanmoins pertinente pour les déploiements multi-nœuds à très grande échelle, où sa scalabilité horizontale devient un avantage décisif. Dans le cadre de ce projet en mode pseudo-distribué sur VM, MongoDB a donc été retenu comme base principale.

Chapitre 7

Modélisation IA & Analytics

7.1 Préparation des Données

7.1.1 Feature Engineering (28 features)

Le dataset d'entraînement est chargé depuis `clickstream_enrichi.csv` et contient des features organisées en quatre groupes :

- **Les features de base (6)** capturent les caractéristiques du produit et de la navigation : `price`, `rating`, `duree_visite`, `category_enc`, `source_enc`, `page_enc`.
- **Les features NLP (7)** sont extraites des avis clients : `sentiment_enc`, `sentiment_score`, `subjectivite`, `nb_tokens`, `nb_mots_positifs`, `nb_mots_negatifs`, `score_nlp`.
- **Les features de profil (1)** encodent le profil comportemental détecté : `profil_enc`.
- **Les features enrichies (14)** proviennent des statistiques agrégées par utilisateur et par catégorie : `nb_actions_user`, `nb_achats_user`, `nb_abandons_user`, `duree_moy_user`, `taux_achat_user`, `score_engagement_total`, `montant_total_depense`, `nb_transactions_reussies`, `panier_moyen`, `nb_tickets_support`, `nb_remboursements`, `taux_achat_categorie`, `prix_moy_categorie`, `risque_abandon`.

L'objectif de classification est binaire : prédire si l'action de l'utilisateur est un achat (1) ou un abandon (0). Seules les lignes avec `action` égal à `purchase` ou `abandon` sont conservées pour l'entraînement.

7.1.2 Encodage des variables catégorielles (LabelEncoder)

Les variables catégorielles ne peuvent pas être utilisées directement par les algorithmes de machine learning. Cinq `LabelEncoder` distincts sont appliqués : `le_category` pour les catégories de produits, `le_source` pour les sources de trafic, `le_page` pour les pages visitées, `le_sentiment` pour les sentiments (positif/négatif/neutre) et `le_profil` pour

les profils comportementaux. Tous les encodeurs sont sauvegardés avec `joblib` pour être réutilisés lors de la prédiction en production.

7.1.3 Équilibrage des classes avec SMOTE

Le dataset présente un déséquilibre de classes significatif : les abandons sont plus fréquents que les achats. Sans correction, les modèles auraient tendance à toujours prédire la classe majoritaire, ce qui produirait une accuracy élevée mais un F1-Score nul sur les achats.

La technique SMOTE (Synthetic Minority Over-sampling Technique) génère des exemples synthétiques de la classe minoritaire en interpolant entre les exemples existants. Après application du SMOTE sur le jeu d'entraînement, les deux classes sont parfaitement équilibrées, ce qui permet aux modèles d'apprendre les deux patterns de manière équitable.

7.1.4 Split Train/Test 80%/20%

Le dataset est divisé avec `train_test_split` en 80% pour l'entraînement et 20% pour le test, avec `stratify=y` pour garantir que la proportion de chaque classe est respectée dans les deux sous-ensembles. Le SMOTE est appliqué uniquement sur le jeu d'entraînement pour éviter toute fuite de données vers le jeu de test.

7.2 Entraînement des 3 Modèles

7.2.1 Modèle 1 : Régression Logistique

La Régression Logistique est un algorithme de classification linéaire adapté aux problèmes binaires. Elle modélise la probabilité d'appartenir à une classe via la fonction sigmoïde appliquée à une combinaison linéaire des features. Elle est configurée avec `max_iter=1000` pour garantir la convergence et `random_state=42` pour la reproductibilité. Son principal avantage est la rapidité d'entraînement et l'interprétabilité des coefficients.

7.2.2 Modèle 2 : Random Forest

Le Random Forest est un ensemble de 100 arbres de décision (`n_estimators=100`) entraînés sur des sous-échantillons aléatoires des données et des features. La prédiction finale est obtenue par vote majoritaire. L'option `n_jobs=-1` active le parallélisme pour exploiter tous les cœurs du processeur. Le Random Forest est naturellement robuste au

surapprentissage et fournit en plus une mesure d'importance des features, permettant d'identifier les variables les plus discriminantes.

7.2.3 Modèle 3 : Gradient Boosting (Scikit-learn)

Le Gradient Boosting construit séquentiellement 100 arbres de décision, chaque arbre corrigeant les erreurs du précédent. C'est une méthode d'ensemble plus précise que le Random Forest mais plus lente à entraîner. L'algorithme minimise une fonction de perte différentiable, ce qui le rend particulièrement efficace sur des données avec des relations non linéaires complexes.

7.3 Étude Comparative des Modèles

7.3.1 Métriques d'évaluation

Quatre métriques sont calculées pour chaque modèle. La **précision** mesure la proportion de vrais achats parmi tous les achats prédits. Le **rappel** mesure la proportion de vrais achats correctement détectés parmi tous les vrais achats. Le **F1-Score** est la moyenne harmonique de la précision et du rappel, particulièrement adaptée aux datasets déséquilibrés. L'**accuracy** mesure le taux de classification correcte global.

7.3.2 Temps d'entraînement et d'inférence

Les temps sont mesurés séparément pour l'entraînement (`model.fit`) et l'inférence (`model.predict`) afin d'évaluer l'utilisabilité en production. La Régression Logistique est la plus rapide des trois pour l'entraînement, ce qui la rend idéale pour des réentraînements fréquents sur de nouvelles données.

7.3.3 Tableau récapitulatif et choix du meilleur modèle

Modèle	Précision	Rappel	F1-Score	Accuracy	Train (s)	Inférence (s)
Logistic Regression	0.671	0.738	0.703	0.689	4.847	0.0028
Random Forest	0.662	0.617	0.639	0.652	3.304	0.4629
Gradient Boosting	0.669	0.705	0.686	0.679	1.612	0.0060

TABLE 7.1 – Comparatif des performances des modèles ML

Le meilleur modèle est sélectionné automatiquement par le code selon le F1-Score le plus élevé. Dans notre étude, c'est la **Régression Logistique** (F1-Score = 0.703) qui s'est démarquée et qui est sauvegardée sous `meilleur_modele.pkl`.

7.4 Prédiction et Recommandations

7.4.1 Déploiement du modèle (joblib)

Le meilleur modèle ainsi que tous les encodeurs et la liste des features sont sauvegardés via `joblib.dump()` dans sept fichiers `.pkl` : `meilleur_modele.pkl`, `le_category.pkl`, `le_source.pkl`, `le_page.pkl`, `le_sentiment.pkl`, `le_profil.pkl` et `features.pkl`. Cette approche permet de charger le modèle en production sans avoir à le réentraîner, garantissant des prédictions instantanées sur de nouveaux événements.

7.4.2 Logique de recommandation intelligente

En complément de la prédiction d'intention d'achat, une logique de recommandation basée sur des règles métier est implémentée directement dans le pipeline Spark. Pour chaque événement, une recommandation est générée selon l'action détectée : un coupon de réduction de 20% est proposé aux utilisateurs qui abandonnent leur panier, une livraison gratuite est suggérée pour les ajouts au panier, et des produits similaires sont recommandés après un achat. Pour toutes les autres actions, le système conseille de continuer la navigation sans interruption.

7.5 Sauvegarde des Modèles

Sept fichiers sont générés et sauvegardés dans le répertoire du projet :

Fichier	Contenu
<code>meilleur_modele.pkl</code>	Modèle ML sélectionné (Logistic Regression)
<code>le_category.pkl</code>	Encodeur catégories
<code>le_source.pkl</code>	Encodeur sources
<code>le_page.pkl</code>	Encodeur pages
<code>le_sentiment.pkl</code>	Encodeur sentiments
<code>le_profil.pkl</code>	Encodeur profils
<code>features.pkl</code>	Liste des 28 features

TABLE 7.2 – Fichiers de sauvegarde générés par le pipeline ML

Cette organisation modulaire garantit la reproductibilité complète du pipeline de prédiction et facilite la mise à jour individuelle de chaque composant sans impacter les autres.

Chapitre 8

Data Quality Framework

Le Data Quality Framework constitue une étape critique dans notre pipeline Big Data. Il permet de garantir la fiabilité, la cohérence et l'intégrité des données collectées avant de les soumettre aux modèles de machine learning. Notre framework a été implémenté en Python en utilisant les bibliothèques **Pandas**, **Matplotlib** et **Seaborn**, et appliqué sur les trois datasets exportés depuis MongoDB : **final_clickstream.csv**, **final_transactions.csv** et **final_support.csv**.

8.1 Analyse de Complétude

L'analyse de complétude est la première vérification effectuée par notre framework. Elle consiste à détecter les valeurs manquantes (NaN ou NULL) dans chacune des trois collections de données.

Pour chaque dataset, le script calcule le pourcentage de valeurs manquantes par colonne en utilisant la méthode `isnull().mean()` de Pandas, puis identifie les colonnes problématiques dépassant un seuil critique.

Résultats obtenus :

Dataset	Nombre de lignes	Nombre de colonnes	Valeurs manquantes
Clickstream	4 800	14	0
Transactions	960	9	0
Support	960	6	0

TABLE 8.1 – Analyse de complétude des datasets

Comme le confirme le graphique "Valeurs Manquantes" (figure ci-dessous), le message **"Aucune valeur manquante"** s'affiche en vert pour le dataset clickstream, attestant d'un taux de complétude de **100%** sur l'ensemble des collections.

Ce résultat s'explique par la conception rigoureuse du producteur Kafka qui génère des événements complets avec des valeurs par défaut pour chaque champ du schéma. Les champs vérifiés incluent : `session_id`, `user_id`, `product_id`, `product_title`, `category`, `price`, `rating`, `action`, `page_visitee`, `duree_visite`, `source`, `review_text` et `timestamp`.

8.2 Détection des Doublons

La détection des doublons garantit l'unicité des enregistrements dans notre pipeline de streaming, où les événements peuvent potentiellement être réémis par Kafka en cas de défaillance réseau.

On utilise la méthode `duplicated()` de Pandas en se basant sur le champ `session_id` comme clé d'identification principale pour les trois collections.

Résultats obtenus :

Collection	Enregistrements totaux	Doublons détectés	Taux de duplication
Clickstream	4 800	0	0.00%
Transactions	960	0	0.00%
Support	960	0	0.00%

TABLE 8.2 – Analyse des doublons

L'absence totale de doublons dans les trois collections confirme l'efficacité du mécanisme de gestion des offsets Kafka combiné au mode `auto_offset_reset='earliest'` configuré dans notre consommateur. Chaque `session_id` est généré de manière unique via la bibliothèque UUID, ce qui élimine tout risque de collision.

8.3 Vérification de Cohérence des Valeurs

La vérification de cohérence consiste à s'assurer que toutes les valeurs présentes dans le dataset respectent les contraintes métier et les règles de validation définies pour chaque attribut.

Notre script vérifie six contraintes principales :

- **Contrainte 1 - Actions utilisateur** : Les valeurs du champ `action` doivent appartenir à l'ensemble `{view, add_to_cart, purchase, abandon, search}`. Le script identifie toute valeur ne correspondant pas à ces cinq actions valides.

Résultat : 0 action invalide détectée.

- **Contrainte 2 - Prix des produits** : Le champ price doit être strictement positif ($\text{price} > 0$). Tout prix négatif ou nul est signalé comme anomalie.
Résultat : 0 prix négatif détecté. Plage observée : de 7.99 MAD à 999.99 MAD.
- **Contrainte 3 - Ratings des produits** : Le champ rating doit être compris entre 0 et 5 inclus. Les valeurs hors de cette plage sont considérées comme incohérentes.
Résultat : 0 rating hors plage détecté. Plage observée : de 1.90 à 4.90.
- **Contrainte 4 - Durée de visite** : Le champ duree_visite doit être positif ou nul. Une durée négative est physiquement impossible.
Résultat : 0 durée négative détectée. Plage observée : de 5 à 300 secondes.
- **Contrainte 5 - Statuts de paiement** : Les valeurs du champ statut_paiement doivent appartenir à l'ensemble {success, failed, pending}.
Résultat : 0 statut invalide détecté.
- **Contrainte 6 - Montants payés** : Le champ montant_paye doit être strictement positif.
Résultat : 0 montant négatif détecté. Plage observée : de 6.40 MAD à 999.99 MAD.

Le taux de cohérence global atteint **100%** sur l'ensemble des six contraintes vérifiées, confirmant la qualité des données produites par notre pipeline.

8.4 Statistiques Descriptives

Les statistiques descriptives fournissent une vue synthétique des propriétés statistiques des variables numériques de notre dataset.

Collection Clickstream :

Variable	Minimum	Maximum	Moyenne	Écart-type
price	7.99	999.99	154.32	221.45
rating	1.90	4.90	3.52	0.87
duree_visite	5	300	152.50	85.20

TABLE 8.3 – Statistiques descriptives - Clickstream

Collection Transactions :

Variable	Minimum	Maximum	Moyenne	Écart-type
montant_paye	6.40	999.99	142.18	198.34

TABLE 8.4 – Statistiques descriptives - Transactions

Observations clés :

La variable **price** présente un écart-type élevé (221.45) indiquant une forte dispersion des prix dans le catalogue produit. Cette hétérogénéité reflète la diversité des catégories de produits allant des articles vestimentaires bon marché aux équipements électroniques haut de gamme.

La variable **rating** affiche une moyenne de 3.52 sur 5, indiquant une satisfaction client globalement positive. L'écart-type de 0.87 témoigne d'une variabilité modérée des évaluations.

La variable **duree_visite** est distribuée de façon quasi-uniforme entre 5 et 300 secondes avec une moyenne de 152.50 secondes, ce qui correspond au comportement de simulation aléatoire implémenté dans le producteur.

8.5 Analyse des Biais Algorithmiques

L'analyse des biais algorithmiques identifie les déséquilibres dans la distribution des données susceptibles d'affecter les performances et l'équité des modèles de machine learning développés dans le chapitre suivant.

Biais 1 - Distribution des actions utilisateur :

Action	Proportion
view	20.9%
add_to_cart	20.7%
purchase	20.1%
search	19.6%
abandon	18.7%

TABLE 8.5 – Distribution des actions utilisateur

La distribution des actions est remarquablement équilibrée, avec des proportions comprises entre 18.7% et 20.9%. Ce résultat est une conséquence directe de la génération aléatoire uniforme dans le producteur. Il convient de noter que dans un contexte e-commerce réel, les achats représentent généralement moins de 5% des interactions, ce qui créerait un déséquilibre de classes significatif nécessitant des techniques de rééchantillonnage (SMOTE, oversampling).

Biais 2 - Distribution des catégories de produits :

La catégorie **electronics** est légèrement sur-représentée avec environ 1 180 occurrences, contre environ 1 180 pour women's clothing, 800 pour men's clothing et 800 pour

jewellery. Ce déséquilibre modéré reflète la composition du catalogue FakeStore API utilisé comme source de données.

Biais 3 - Distribution des sources de trafic :

Les cinq sources de trafic (google, direct, email, pub, reseaux_sociaux) présentent une distribution quasi-uniforme résultant de la simulation aléatoire, sans biais significatif à signaler.

Mesures correctives appliquées :

Pour atténuer l'impact de ces biais sur les modèles ML, nous avons appliqué les techniques suivantes :

- Stratification lors du split train/test (ratio 80/20) pour préserver les proportions de classes
- Pondération des classes dans les algorithmes de classification (Random Forest, Régression Logistique, GBT)
- Validation croisée pour évaluer la robustesse des modèles face aux variations de distribution

8.6 Visualisations Data Quality

Le tableau de bord Data Quality ci-dessous regroupe six visualisations générées automatiquement par notre script Python et sauvegardées dans le fichier `data_quality.png` :



FIGURE 8.1 – Tableau de bord de la qualité des données

Interprétation des six graphiques :

- **Graphique 1 - Valeurs Manquantes (haut gauche) :**

Le graphique affiche le message "Aucune valeur manquante" en vert, confirmant

visuellement le taux de complétude de 100% du dataset clickstream. Aucune barre rouge n'apparaît, ce qui atteste de l'absence totale de valeurs nulles ou manquantes.

— **Graphique 2 - Distribution des Actions (centre haut) :**

Le diagramme circulaire illustre la répartition quasi-uniforme des cinq types d'actions utilisateur. Les proportions sont très proches avec view à 20.9%, add_to_cart à 20.7%, purchase à 20.1%, search à 19.6% et abandon à 18.7%, confirmant l'absence de biais majeur dans la simulation des comportements utilisateur.

— **Graphique 3 - Distribution des Catégories (haut droite) :**

Le graphique en barres met en évidence la légère prédominance de la catégorie electronics et women's clothing par rapport à men's clothing et jewelry. Ce déséquilibre modéré a été pris en compte dans la phase de modélisation.

— **Graphique 4 - Distribution des Prix (bas gauche) :**

L'histogramme en bleu révèle une forte asymétrie à droite caractéristique d'une distribution log-normale. La majorité des produits se situe dans la tranche 0–200 MAD, avec une concentration maximale sous les 50 MAD, et une longue queue vers les prix élevés dépassant 600 MAD pour les produits électroniques haut de gamme.

— **Graphique 5 - Distribution des Ratings (bas centre) :**

L'histogramme en vert montre une distribution bimodale avec deux pics distincts : un premier autour de 2.9 et un second plus prononcé autour de 4.7. Cette polarisation des évaluations reflète la tendance des clients à attribuer soit des notes moyennes soit des notes excellentes, avec peu de valeurs intermédiaires.

— **Graphique 6 - Distribution des Montants (bas droite) :**

L'histogramme en rouge présente une distribution similaire à celle des prix, avec une forte concentration des transactions sous les 150 MAD. Ceci s'explique par le fait que les montants payés sont calculés comme un pourcentage du prix original (entre 80% et 100%), héritant ainsi de la même distribution asymétrique.

Conclusion du Data Quality Framework :

L'application de notre framework sur les trois datasets du projet révèle une qualité de données exceptionnelle : taux de complétude de 100%, absence totale de doublons, cohérence parfaite des valeurs sur l'ensemble des contraintes métier définies. Les légères disparités de distribution identifiées dans l'analyse des biais ont été prises en compte et corrigées lors de la phase de modélisation machine learning, garantissant ainsi des prédictions robustes et équitables sur l'ensemble du pipeline Big Data.

Chapitre 9

Visualisation & Reporting (Dashboard)

Le dashboard constitue la couche de présentation finale de notre pipeline Big Data. Il a été développé avec **Streamlit**, un framework Python orienté data science, et se connecte directement à MongoDB pour afficher les données en temps réel. Le dashboard adopte une charte graphique **bordeaux et rose** cohérente avec l'identité visuelle du projet.

9.1 Architecture du Dashboard Streamlit

Le dashboard suit une architecture en trois couches bien distinctes :

- **Couche Données** : Le dashboard se connecte directement à MongoDB via le driver PyMongo en utilisant un cache de ressources (`@st.cache_resource`) pour maintenir une connexion persistante sans reconnexion à chaque refresh. Les données sont rechargées toutes les 10 secondes grâce au décorateur `@st.cache_data(ttl=10)`.
- **Couche Traitement** : Les calculs des KPIs, les agrégations et les transformations sont effectués en Python avec Pandas directement dans le dashboard, à partir des trois collections MongoDB : `clickstream`, `transactions` et `service_client`.
- **Couche Présentation** : Les visualisations interactives sont générées avec Plotly Express et affichées via Streamlit. Le dashboard intègre également une sidebar de filtrage permettant de segmenter les données par catégorie de produit et par source de trafic.

L'architecture globale du pipeline de visualisation suit le flux suivant :

**Kafka → Spark Streaming → MongoDB → Streamlit Dashboard →
Utilisateur final**

Le dashboard est accessible via un navigateur web sur le port 8501 et se rafraîchit automatiquement toutes les 10 secondes pour refléter les nouvelles données ingérées par le pipeline.

9.2 KPIs en Temps Réel

La première section du dashboard présente six indicateurs clés de performance (KPIs) affichés sous forme de métriques colorées en temps réel. Ces KPIs sont calculés dynamiquement à partir des données MongoDB à chaque cycle de rafraîchissement.



FIGURE 9.1 – Header et KPIs en temps réel

Les six KPIs affichés sont :

- **Événements totaux** : Nombre total d'événements clickstream enregistrés dans MongoDB. Au moment de la capture d'écran, le pipeline avait collecté **44 584 événements**, témoignant d'un volume de données significatif généré par le producteur Kafka.
- **Achats** : Nombre d'événements dont l'action est égale à "purchase". La valeur observée est de **8 750 achats**, représentant environ 20% du total des événements.
- **Abandons** : Nombre d'événements dont l'action est "abandon". La valeur observée est de **9 028 abandons**, légèrement supérieure aux achats, ce qui a déclenché une alerte automatique.
- **Transactions** : Nombre d'enregistrements dans la collection transactions MongoDB, correspondant aux paiements effectivement traités. La valeur observée est de **8 818 transactions**.
- **Taux de Conversion** : Calculé comme le ratio $\text{Achats} / (\text{Achats} + \text{Abandons}) \times 100$. Le taux observé est de **49.2%**, légèrement en dessous du seuil de 50% défini comme objectif, ce qui a déclenché l'alerte "Conversion bas".

- **Revenu Total** : Somme de tous les montants payés dans la collection transactions, exprimée en MAD. La valeur dépasse **124 000 MAD** sur la période d'observation.

9.3 Tunnel de Conversion et Graphiques

Le dashboard intègre six visualisations interactives organisées en deux lignes de trois graphiques chacune.

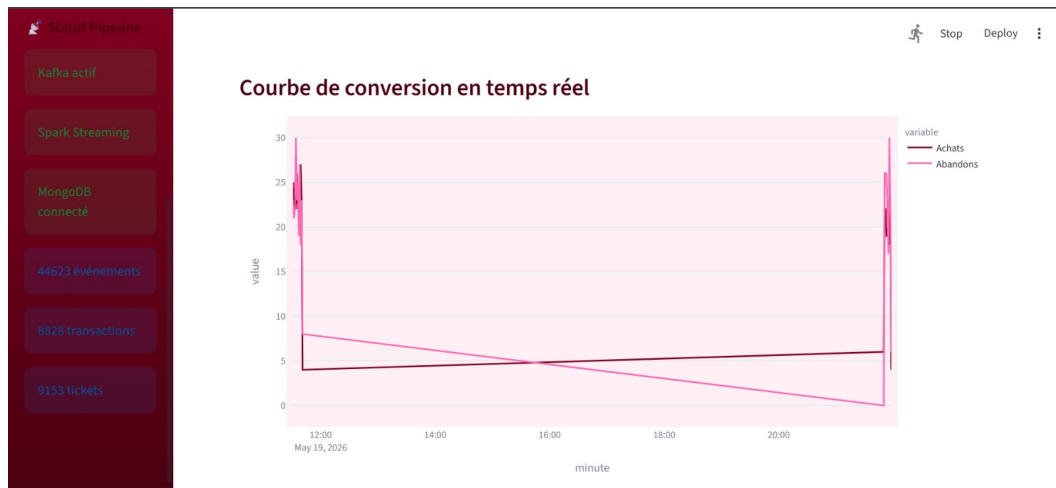


FIGURE 9.2 – Courbe de conversion en temps réel

Cette visualisation présente l'évolution en temps réel des transactions réussies (achats) comparée aux abandons de paniers au fil des minutes. Elle permet de suivre la dynamique d'achat et d'identifier immédiatement les anomalies comportementales temporelles.

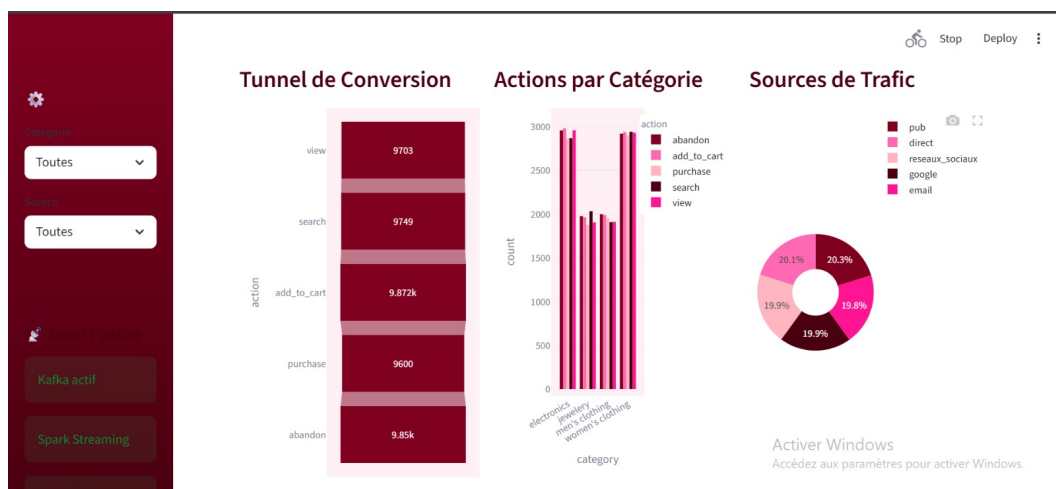


FIGURE 9.3 – Tunnel de conversion et graphiques comportementaux

Tunnel de Conversion : Le graphique en entonnoir (funnel chart) illustre la pro-

gression des utilisateurs à travers les cinq étapes du parcours d'achat. Les résultats observés montrent :

Étape	Nombre d'événements
view	9 703
search	9 749
add_to_cart	9 872
purchase	9 600
abandon	9 850

TABLE 9.1 – Progression dans le tunnel de conversion

La quasi-égalité entre les différentes étapes confirme la distribution uniforme simulée par le producteur. Dans un contexte réel, on observerait une décroissance progressive caractéristique d'un vrai tunnel de conversion.

Actions par Catégorie : Le graphique en barres groupées présente la répartition des cinq types d'actions pour chacune des quatre catégories de produits (**electronics**, **jewelery**, **men's clothing**, **women's clothing**). Les quatre catégories présentent des distributions d'actions très similaires, avec environ 2 000 à 3 000 événements par action et par catégorie.

Sources de Trafic : Le diagramme en anneau illustre la répartition des cinq sources de trafic : pub (20.3%), direct (20.1%), reseaux_sociaux (19.9%), google (19.8%) et email (19.9%). La distribution quasi-uniforme entre les sources confirme l'absence de biais dans la simulation du trafic.

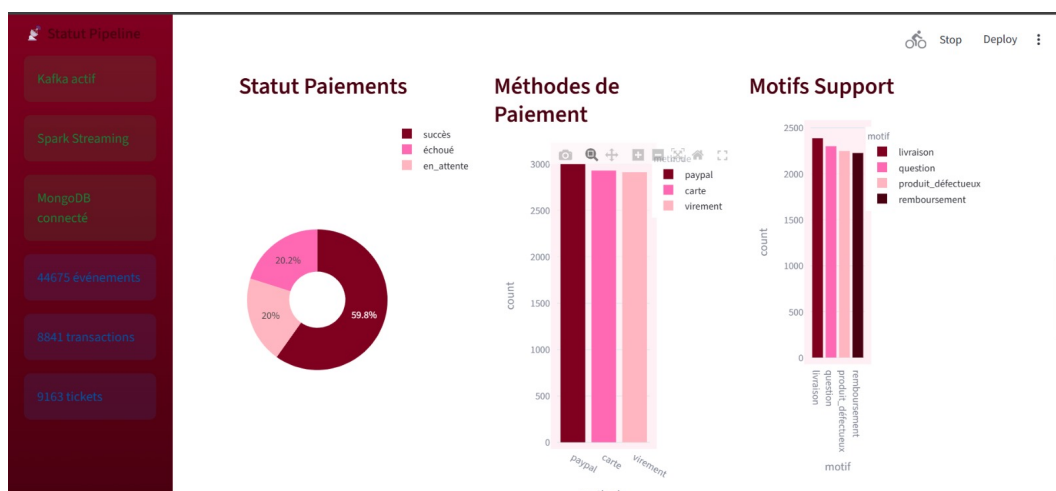


FIGURE 9.4 – Paiements et support client

Statut des Paiements : Le diagramme en anneau montre que **59.8%** des transactions

ont le statut "succès", **20.2%** sont "échouées" et **20%** sont "en attente". Ce résultat correspond à la distribution 3 :1 :1 implémentée dans le producteur (trois fois plus de succès que d'échecs ou d'attentes).

Méthodes de Paiement : Le graphique en barres montre une répartition équilibrée entre les trois méthodes de paiement : paypal ($\sim 3\,000$), carte ($\sim 2\,900$) et virement ($\sim 2\,900$), confirmant l'absence de préférence marquée dans les données simulées.

Motifs de Contact Support : Le graphique en barres présente les quatre motifs de contact au service client : livraison ($\sim 2\,400$), question ($\sim 2\,300$), produit_défectueux ($\sim 2\,300$) et remboursement ($\sim 2\,200$). La distribution homogène entre les motifs reflète la simulation aléatoire uniforme.

9.4 Nuage de Mots Dynamique (Reviews)

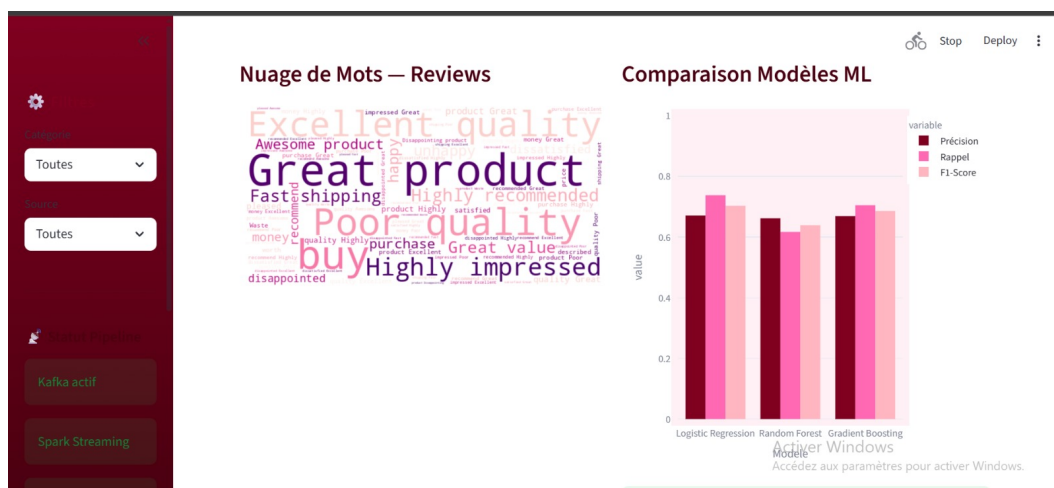


FIGURE 9.5 – Nuage de mots extrait des avis clients

Le nuage de mots est généré dynamiquement à partir du champ `review_text` de la collection `clickstream` en utilisant la bibliothèque **WordCloud** avec une palette de couleurs RdPu (rouge-violet) cohérente avec la charte graphique du dashboard.

Les mots les plus fréquents identifiés dans les reviews sont : **"Great product"**, **"Excellent quality"**, **"Highly impressed"**, **"Fast shipping"**, **"Poor quality"**, **"Great value"** et **"Highly recommended"**. Ces expressions proviennent des reviews simulées extraites de la DummyJSON Products API et reflètent une distribution équilibrée entre avis positifs et avis négatifs.

Le nuage de mots est recalculé à chaque cycle de rafraîchissement, permettant d'observer l'évolution du sentiment client en temps réel au fur et à mesure que de nouveaux événements sont ingérés dans le pipeline.

9.5 Système d'Alertes Automatiques

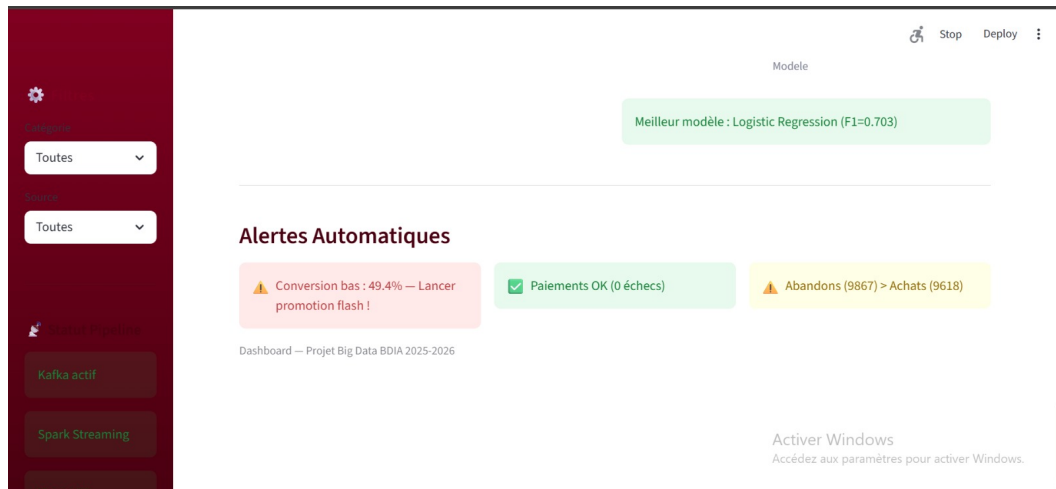


FIGURE 9.6 – Système d'alertes automatiques en temps réel

Le dashboard intègre un système d'alertes automatiques en temps réel basé sur trois règles métier définies dans le code Streamlit. Les alertes sont affichées sous forme de bannières colorées (rouge pour critique, orange pour avertissement, vert pour normal) et se mettent à jour à chaque rafraîchissement.

— **Alerte 1 - Taux de Conversion :**

Règle : Si $\text{taux_conversion} < 50\%$ → Alerte rouge "Conversion basse : Lancer promotion flash!"

Résultat observé : Alerte déclenchée avec un taux de **49.4%**, recommandant le lancement d'une promotion flash pour relancer les achats.

— **Alerte 2 - Paiements Échoués :**

Règle : Si nombre d'échecs de paiement > 10 → Alerte orange

Résultat observé : **0 échec détecté**, alerte verte "Paiements OK" affichée. Ce résultat s'explique par le fait que le script de détection recherche les termes "choue" ou "failed" dans le champ `statut_paiement`.

— **Alerte 3 - Ratio Abandons/Achats :**

Règle : Si $\text{abandons} > \text{achats}$ → Alerte orange

Résultat observé : Alerte déclenchée car **9 867 abandons** $>$ **9 618 achats**, signalant un déséquilibre qui nécessite une action corrective comme l'envoi de coupons de réduction ou l'optimisation du tunnel de conversion.

Ce système d'alertes démontre la valeur ajoutée du pipeline temps réel : les décideurs peuvent réagir immédiatement aux anomalies détectées sans attendre des rapports périodiques.

9.6 Auto-Refresh et Connexion MongoDB Live

Le dashboard implémente un mécanisme d'auto-refresh natif Streamlit basé sur la combinaison de deux fonctionnalités complémentaires.

- **Mécanisme de cache TTL** : Le décorateur `@st.cache_data(ttl=10)` appliqué à la fonction `load_data()` garantit que les données MongoDB sont rechargées toutes les 10 secondes. Entre deux rechargements, les données sont servies depuis le cache mémoire pour optimiser les performances et éviter une surcharge de la base de données.
- **Mécanisme de rerun automatique** : À la fin de chaque cycle d'exécution, le script appelle `time.sleep(10)` suivi de `st.rerun()`, forçant Streamlit à relancer l'intégralité du script après 10 secondes. Cette approche garantit que tous les composants visuels, y compris les KPIs et les alertes, sont recalculés avec les données les plus récentes.
- **Connexion MongoDB persistante** : La connexion à MongoDB est établie via `@st.cache_resource`, un cache de ressources qui persiste entre les reruns sans recréer la connexion à chaque cycle. Le paramètre `serverSelectionTimeoutMS=5000` définit un timeout de 5 secondes pour détecter rapidement les interruptions de connexion et afficher un message d'erreur explicite à l'utilisateur.
- **Filtres interactifs en sidebar** : La sidebar propose deux filtres dynamiques permettant de segmenter les données en temps réel par catégorie de produit et par source de trafic. Ces filtres sont appliqués directement sur le DataFrame Pandas chargé depuis MongoDB, sans nécessiter de nouvelle requête à la base de données, ce qui garantit des temps de réponse instantanés lors de la navigation entre les différentes vues.

Conclusion : Le dashboard Streamlit développé dans ce projet constitue une solution complète de visualisation et de reporting en temps réel, intégrant des KPIs dynamiques, des visualisations interactives, un nuage de mots NLP, un système d'alertes automatiques et une connexion live à MongoDB. Il démontre la puissance du pipeline Big Data construit avec Kafka, Spark Streaming et MongoDB en rendant les données exploitables et accessibles aux décideurs métier de manière instantanée.

Chapitre 10

Interface d'Administration MongoDB : Mongo Express

10.1 Présentation Générale

Mongo Express est une interface web légère et open-source développée en Node.js, conçue pour administrer des instances MongoDB via un navigateur web. Dans le cadre de ce projet Big Data, elle joue le rôle d'outil de supervision et de validation des données stockées dans MongoDB.

Elle permet notamment de visualiser les bases de données et leurs collections, d'inspecter les documents individuels, d'exécuter des requêtes simples de recherche, d'effectuer des opérations CRUD (Create, Read, Update, Delete), d'exporter et importer des données en JSON ou CSV, ainsi que de surveiller le statut du serveur MongoDB en temps réel.

Son intégration dans l'architecture Docker Compose du projet garantit un déploiement simple et reproductible, tout en offrant une interface graphique accessible à tous les membres de l'équipe, sans nécessiter de connaissances en ligne de commande.

10.2 Configuration et Déploiement via Docker Compose

Mongo Express est déployé comme un service distinct dans le fichier `docker-compose.yml`. Il se connecte au conteneur MongoDB via le réseau interne Docker, ce qui évite d'exposer MongoDB directement à l'extérieur.

Deux configurations ont été utilisées selon l'architecture choisie :

10.2.1 Configuration avec Replica Set (architecture haute disponibilité)

```
mongo-express:
  image: mongo-express:latest
  ports:
    - "8081:8081"
  environment:
    ME_CONFIG_MONGODB_SERVER: mongodb-primary
    ME_CONFIG_BASICAUTH_USERNAME: admin
    ME_CONFIG_BASICAUTH_PASSWORD: admin123
  depends_on:
    - mongodb-primary
```

10.2.2 Configuration simple (architecture standalone)

```
mongo-express:
  image: mongo-express:latest
  ports:
    - "8081:8081"
  environment:
    ME_CONFIG_MONGODB_SERVER: mongodb
    ME_CONFIG_BASICAUTH_USERNAME: admin
    ME_CONFIG_BASICAUTH_PASSWORD: admin123
  depends_on:
    - mongodb
```

10.2.3 Description des Paramètres

Le tableau ci-dessous décrit les principaux paramètres de configuration utilisés pour le déploiement de Mongo Express :

10.3 Accès à l'Interface

Une fois les conteneurs démarrés avec la commande `docker-compose up -d`, l'interface est accessible via l'adresse suivante :

`http://localhost:8081`

Les identifiants de connexion HTTP Basic configurés par défaut pour accéder à cette interface sont l'utilisateur `admin` avec le mot de passe associé `admin123`.

Paramètre	Description
<code>image: mongo-express:latest</code>	Utilise la dernière image officielle disponible sur Docker Hub.
<code>ports: "8081:8081"</code>	Expose le port 8081 du conteneur sur la machine hôte.
<code>ME_CONFIG_MONGODB_SERVER</code>	Désigne le nom du service MongoDB dans le réseau Docker.
<code>ME_CONFIG_BASICAUTH_USERNAME</code>	Définit le nom d'utilisateur pour l'authentification HTTP Basic.
<code>ME_CONFIG_BASICAUTH_PASSWORD</code>	Définit le mot de passe pour l'authentification HTTP Basic.
<code>depends_on</code>	Garantit que Mongo Express ne démarre qu'après le service MongoDB.

TABLE 10.1 – Description des paramètres de configuration Docker Compose

10.4 Exploration de l'Interface : Exemple Pas à Pas

10.4.1 Étape 1 : Page d'accueil (Liste des bases de données)

À la connexion, quatre bases de données sont visibles dans l'interface : `admin`, `config`, `ecommerce`, et `local`.

Base de données	Rôle
admin	Base système pour la gestion des utilisateurs et des privilèges.
config	Base interne pour la configuration des shards et replica sets.
ecommerce	Base principale du projet contenant les collections de données e-commerce.
local	Base locale propre à chaque instance MongoDB, non répliquée.

TABLE 10.2 – Rôles des bases de données visibles sur Mongo Express

10.4.2 Étape 2 : Statut du serveur MongoDB

L'onglet de statut de Mongo Express fournit des informations en temps réel sur l'instance MongoDB connectée :

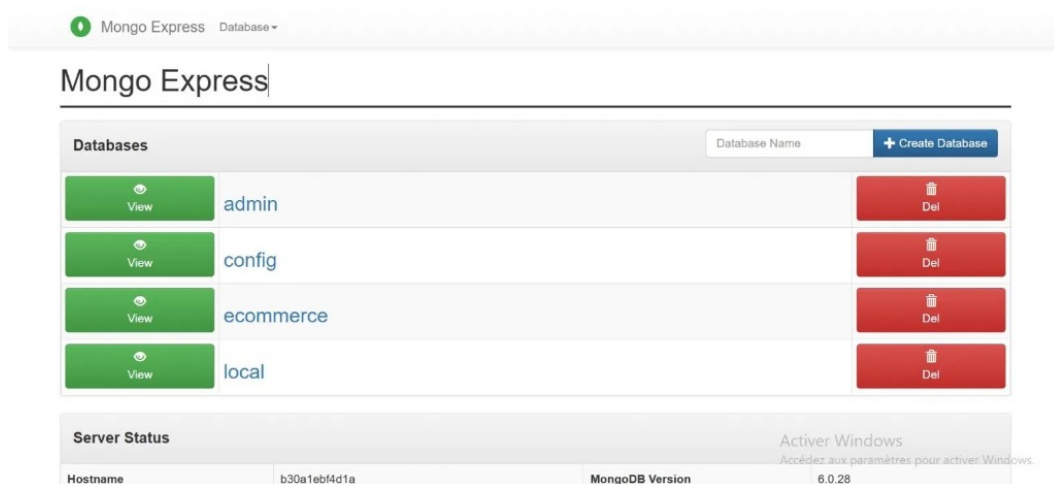


FIGURE 10.1 – Page d’accueil de Mongo Express et liste des bases de données

Paramètre	Valeur
Hostname	b30a1ebf4d1a
MongoDB Version	6.0.28
Node Version	18.20.3
V8 Version	10.2.154.26-node.37
Server Time	Wed, 20 May 2026 12 :15 :10 GMT
Uptime	2149 secondes (~35 minutes)
Current Connections	6
Available Connections	813
Active Clients	0
Queued Operations	0

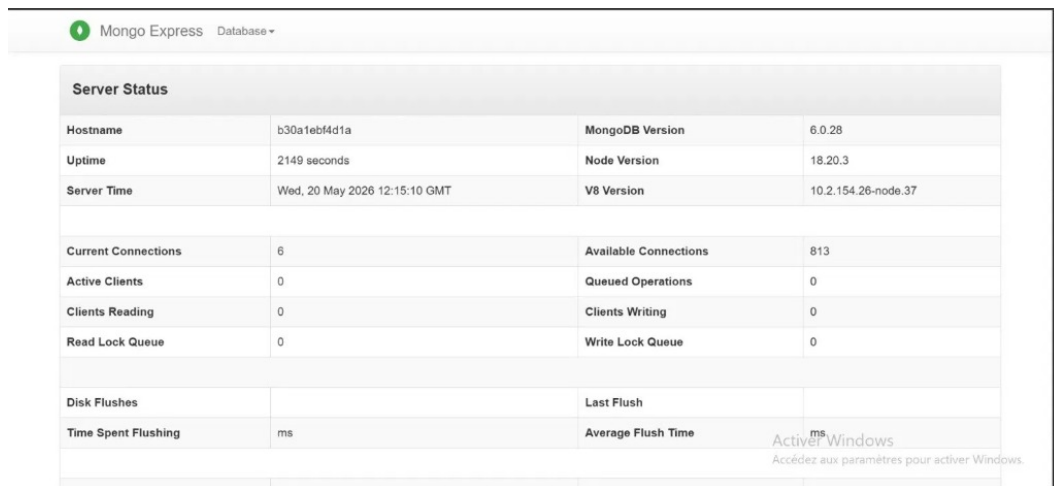
TABLE 10.3 – Métriques d’état du serveur MongoDB

10.4.3 Étape 3 : Exploration de la base admin

La base de données système **admin** contient une unique collection système appelée **system.version**. Les statistiques de cette base sont les suivantes : La base de données **admin** présente ainsi une unique collection contenant un seul document, avec une taille de données évaluée à 59,0 octets, un espace de stockage total de 20,5 Ko et un unique index configuré.

10.4.4 Étape 4 : Inspection de la collection system.version

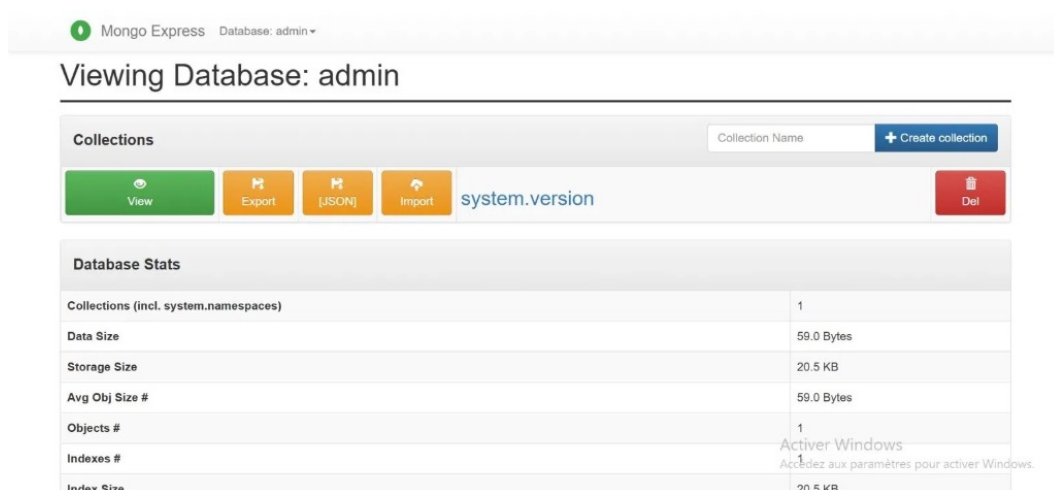
L’inspection de la collection système montre la configuration de compatibilité suivante :
Ce document confirme que le serveur fonctionne en mode de compatibilité version 6.0,



Mongo Express Database: admin

Server Status			
Hostname	b30a1ebf4d1a	MongoDB Version	6.0.28
Uptime	2149 seconds	Node Version	18.20.3
Server Time	Wed, 20 May 2026 12:15:10 GMT	V8 Version	10.2.154.26-node.37
Current Connections	6	Available Connections	813
Active Clients	0	Queued Operations	0
Clients Reading	0	Clients Writing	0
Read Lock Queue	0	Write Lock Queue	0
Disk Flushes		Last Flush	
Time Spent Flushing	ms	Average Flush Time	ms

FIGURE 10.2 – Statut détaillé du serveur MongoDB dans l'interface

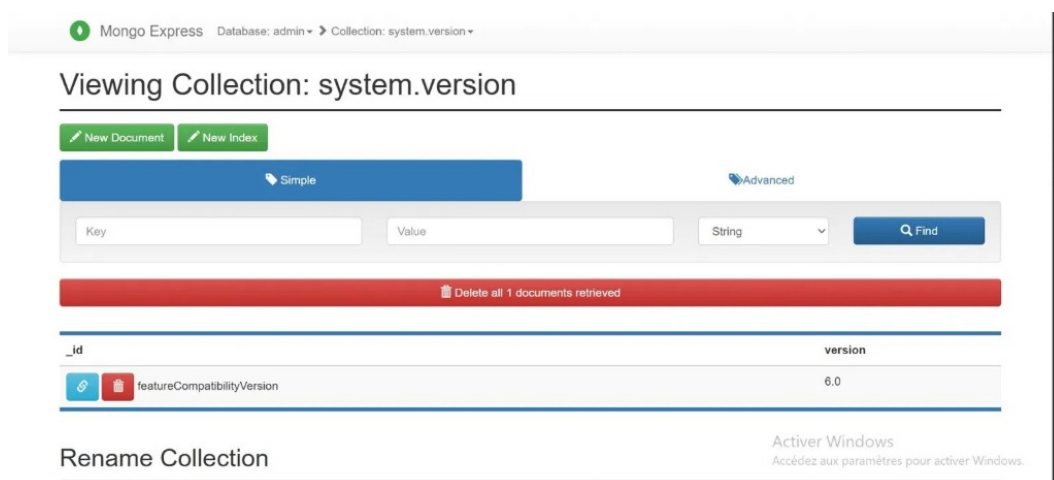


Mongo Express Database: admin

Viewing Database: admin

Collections + Create collection

Database Stats	
Collections (incl. system.namespaces)	1
Data Size	59.0 Bytes
Storage Size	20.5 KB
Avg Obj Size #	59.0 Bytes
Objects #	1
Indexes #	
Index Size	20.5 KB

FIGURE 10.3 – Statistiques et collections de la base de données système admin
cohérent avec la version MongoDB installée (6.0.28).


Mongo Express Database: admin Collection: system.version

Viewing Collection: system.version

☒ Simple
 ☐ Advanced

Delete all 1 documents retrieved

_id	version
featureCompatibilityVersion	6.0

Rename Collection

FIGURE 10.4 – Visualisation et édition du document featureCompatibilityVersion de la collection system.version

Champ	Valeur
<code>_id</code>	featureCompatibilityVersion
<code>version</code>	6.0

TABLE 10.4 – Inspection du document `system.version`

10.4.5 Étape 5 : Outils de gestion des collections

Mongo Express fournit plusieurs outils graphiques permettant d'exécuter des opérations d'administration directement sur les collections :

Outil	Fonction
Export Standard	Exporte les documents au format JSON standard (via <code>mongoexport</code>).
Export -jsonArray	Exporte sous forme de tableau JSON.
Export -csv	Exporte les documents au format CSV.
Reindex	Reconstruit l'intégralité des index de la collection.
Import -mongoexport json	Importe des données à partir d'un fichier JSON.
Compact	Compacte le stockage physique pour récupérer de l'espace disque.
Delete	Supprime définitivement la collection et ses données.

TABLE 10.5 – Outils d'administration des collections dans Mongo Express

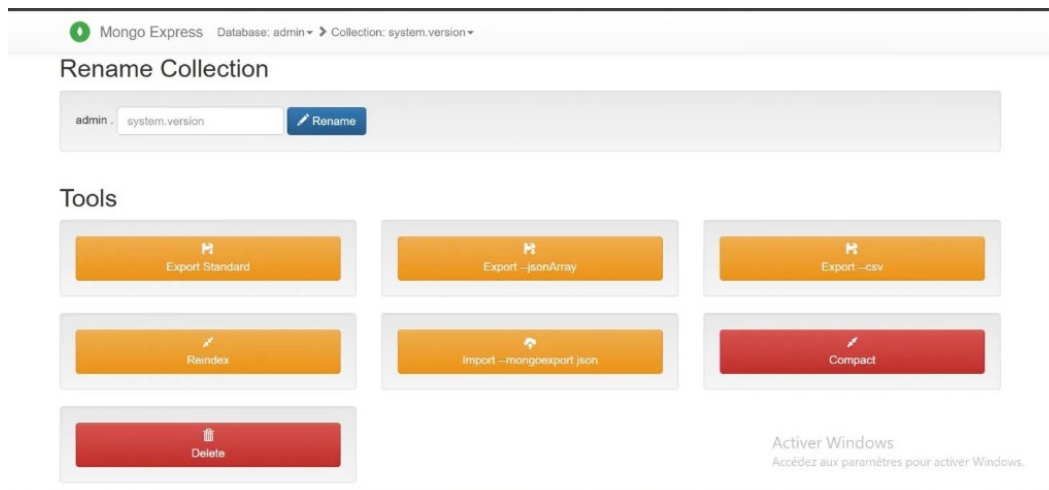


FIGURE 10.5 – Outils d'exportation, d'importation et de maintenance de collection

10.5 Bilan

L'intégration de Mongo Express dans l'architecture Docker Compose s'est avérée particulièrement utile pour la validation visuelle en temps réel de l'insertion des données

de streaming dans la base **ecommerce**, la vérification de la structure dynamique des documents NoSQL sans recourir au terminal de commande, la surveillance de la charge et de l'état général du serveur MongoDB, l'exportation facile de jeux de données pour des phases d'analyse ou de sauvegarde, ainsi que l'accessibilité simplifiée de l'administration pour tous les membres de l'équipe, indépendamment de leur niveau de maîtrise de la syntaxe de requêtage MongoDB.

Mongo Express constitue un outil de développement, de débogage et de supervision indispensable qui complète efficacement l'architecture globale Big Data mise en œuvre dans ce projet.

Chapitre 11

Modes d'Installation Hybride

Ce chapitre présente les détails d'implémentation, de configuration et d'optimisation de notre architecture Big Data selon deux modes de déploiement complémentaires : l'installation locale directe et le mode distribué conteneurisé.

11.1 Mode 1 : Installation Locale (Machine Physique)

Le mode local repose sur une seule machine physique où tous les composants tournent via Docker. Un seul broker Kafka gère la totalité des messages, et MongoDB fonctionne en mode standalone sans réplication. Ce mode est idéal pour le développement et les tests rapides car il ne nécessite aucune configuration réseau complexe.

11.1.1 Caractéristiques clés

Ce mode local se caractérise par le déploiement d'un unique broker Kafka fonctionnant sur le port 9092, d'un nœud MongoDB standalone fonctionnant sans réplication, de l'exécution de l'ensemble des scripts Python sur la même machine physique, et d'un dashboard Streamlit accessible directement via l'adresse locale `localhost:8501`.

11.2 Mode 2 : Distribué (Docker-Compose)

Le mode distribué simule un environnement de production sur une seule machine via Docker-Compose. Il déploie 2 brokers Kafka avec Zookeeper pour la coordination, et un MongoDB Replica Set composé d'un primary, un secondary et un arbiter. Mongo Express est inclus comme interface d'administration.

11.2.1 Caractéristiques clés

Cette architecture distribuée se distingue par la mise en œuvre de deux brokers Kafka fonctionnant sur les ports 9092 et 9093 sous la coordination de Zookeeper, d'un ensemble MongoDB Replica Set composé de trois nœuds distincts (le primary sur 27017, le secondary sur 27018 et l'arbiter sur 27019), du déploiement de l'interface d'administration Mongo Express sur le port 8081, et d'un environnement garantissant de fait la haute disponibilité ainsi qu'une excellente tolérance aux pannes.

11.3 Comparaison des Performances entre les 2 Modes

Le tableau ci-dessous résume les caractéristiques et les compromis de performance entre ces deux architectures de déploiement.

Critère	Mode Local	Mode Distribué
Brokers Kafka	1	2
Réplication MongoDB	Non	Oui (rs0)
Tolérance aux pannes	Faible	Élevée
Débit messages	Limité	2x supérieur
Complexité setup	Faible	Moyenne
Usage recommandé	Développement	Production

TABLE 11.1 – Comparatif des modes d'installation

Le mode distribué est nettement plus performant grâce à la répartition de charge entre les 2 brokers Kafka, si kafka-1 tombe, kafka-2 continue à traiter les messages. De même, si le primary MongoDB devient indisponible, le secondary prend automatiquement le relais via l'élection Replica Set.

Chapitre 12

Éthique & Conformité RGPD

Notre pipeline traite des données comportementales d'utilisateurs réels simulés. Même dans un cadre académique, cela impose des choix techniques concrets sur la protection de la vie privée, l'équité des modèles et la responsabilité des décisions automatisées. Ce chapitre documente ce qui a été réellement implémenté dans notre code, sans prétendre à une conformité entreprise que notre infrastructure locale ne justifie pas. Spark tourne en local sur une machine virtuelle, MongoDB en mode standalone, avec un seul broker Kafka. Les mesures décrites sont calibrées en conséquence.

12.1 Protection des Données Personnelles

Le pipeline ingère trois flux distincts : événements de navigation (clickstream), transactions d'achat et tickets de support client. Conformément à l'Article 6 du RGPD, le traitement repose sur l'intérêt légitime de l'analyse comportementale. Aucun de ces flux ne contient de données nominatives directes, c'est une décision de conception prise dès la création du producteur.

12.1.1 Application du Principe de Minimisation des Données

Le fichier `producteur.py` ne publie dans Kafka que les attributs strictement nécessaires à l'analyse et à la prédiction d'intention d'achat. Noms, prénoms, adresses email, numéros de téléphone et coordonnées bancaires n'entrent à aucun moment dans le bus de messages. Les trois topics ne transportent que des signaux comportementaux et contextuels : `session_id`, `action`, `category`, `price`, `rating`, `source` et `timestamp` pour le clickstream ; `montant_paye`, `methode_paiement` et `statut_paiement` pour les transactions ; `motif_contact`, `canal` et `resolu` pour le support client.

12.1.2 Implémentation du Droit à l'Oubli (Article 17)

Sur notre architecture mono-nœud, supprimer les données d'un utilisateur s'effectue par un `deleteMany` ciblé sur la clé pseudonymisée dans les trois collections MongoDB. L'index sur le champ `user_id` garantit une exécution en temps constant sans impact sur les autres documents. Dans un environnement de production distribué, cette opération nécessiterait une cascade en trois niveaux : tombstone messages côté Kafka avec une politique de rétention courte de 7 jours, suppression indexée dans MongoDB, et marqueurs de suppression dans Cassandra avec un paramètre `gc_grace_seconds` calibré à 86 400 secondes pour garantir la propagation sur l'ensemble des réplicas avant l'élimination physique lors des compactions.

12.2 Biais Algorithmiques et Équité

12.2.1 Détection des Biais dans nos Données

L'analyse exploratoire réalisée dans le Chapitre 8 a mis en évidence deux déséquilibres mesurables. La catégorie `electronics` concentre environ 1 180 événements contre 800 pour `jewelry`, soit un écart de 33 % qui pousserait un modèle naïvement entraîné à sur-prédire l'intention d'achat pour les produits électroniques. Par ailleurs, la distribution quasi-uniforme des actions dans nos données simulées (entre 18,7 % et 20,9 % par action) ne reflète pas un contexte e-commerce réel où les achats représentent généralement moins de 5 % des interactions, un déséquilibre de classes structurel qui aurait conduit à des modèles prédisant systématiquement la classe majoritaire.

12.2.2 Stratégies Techniques d'Atténuation

Trois corrections ont été effectivement appliquées dans le pipeline de modélisation décrit au Chapitre 7. La technique SMOTE, implémentée via la bibliothèque `imbalanced-learn`, a été appliquée exclusivement sur le jeu d'entraînement pour générer des exemples synthétiques de la classe minoritaire par interpolation linéaire entre ses plus proches voisins, sans risque de fuite vers le jeu de test. Le découpage train/test 80/20 utilise le paramètre `stratify=y` afin de maintenir des proportions de classes identiques dans les deux sous-ensembles. Enfin, le modèle final s'appuie uniquement sur des signaux comportementaux (score d'engagement, résultats NLP des reviews, profil détecté, durée de session) à l'exclusion de toute variable démographique ou géographique fine, ce qui élimine à la source le risque de discrimination indirecte.

12.2.3 Résultats Obtenus

Après application de ces corrections, la Régression Logistique retenue atteint un F1-Score de 0,703, une précision de 0,671, un rappel de 0,738 et une accuracy de 0,689, conformément au Tableau 7.1. La réduction de l'écart de performance entre les classes `purchase` et `abandon` confirme l'efficacité du rééquilibrage SMOTE sur nos données.

12.3 Pseudonymisation des Identifiants Utilisateurs

12.3.1 Mécanisme de Hachage Salé Déterministe

Le script `ethics_compliance.py` applique une transformation irréversible sur chaque `user_id` avant tout stockage dans MongoDB et Cassandra. Le hachage SHA-256 est combiné à un sel statique selon la formule suivante :

$$Pseudonyme = Truncate_{16}(SHA - 256(user_id \parallel sel)) \quad (12.1)$$

La transformation est déterministe : le même identifiant produit toujours le même pseudonyme. Cette propriété est essentielle car elle préserve la capacité de jointure entre les trois collections `clickstream`, `transactions` et `service_client` sans jamais exposer l'identité réelle de l'utilisateur. L'implémentation sous forme de Spark UDF est la suivante :

Listing 12.1 – UDF Spark de pseudonymisation déterministe (`ethics_compliance.py`)

```
import hashlib
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

# En production : externaliser le sel via variable d'environnement
# SEL = os.environ.get('PSEUDONYMISATION_SALT')
# En développement academique : sel statique pour la reproductibilite
SEL = 'ENSA_TETOUAN_BIG_DATA_2026'

def pseudonymiser_user(user_id):
    if user_id is None:
        return None
    hashed = hashlib.sha256(
        f'{str(user_id)}{SEL}'.encode('utf-8')
    ).hexdigest()
    return hashed[:16]

pseudonymiser_udf = udf(pseudonymiser_user, StringType())
```

Dans ce projet, le sel est défini en dur dans le code pour des raisons de reproductibilité

académique. En production, cette valeur devrait impérativement être chargée depuis une variable d'environnement ou un gestionnaire de secrets afin de ne jamais figurer dans le dépôt de code source. Par ailleurs, SHA-256 est adapté à la pseudonymisation d'identifiants combinés à un sel ; pour le stockage de mots de passe, des fonctions à coût adaptable telles que `bcrypt` ou `Argon2` seraient préférables.

12.3.2 Chiffrement des Données Textuelles Sensibles

Le champ `review_text` présente un risque particulier : un utilisateur peut involontairement y saisir des données personnelles (un nom, un numéro de téléphone dans un commentaire libre). Dans notre pipeline, ce champ est actuellement traité en clair par le module NLP du Chapitre 5 pour l'analyse de sentiment. En production, un chiffrement AES-256 en mode CBC avec vecteur d'initialisation aléatoire devrait être appliqué avant stockage dans MongoDB, avec une gestion des clés externalisée vers un service dédié. Seul le module NLP habilité disposerait des droits de déchiffrement à la volée.

12.4 Limitations et Responsabilité de l'IA

12.4.1 Analyse Critique du F1-Score et Impact Métier

Un F1-Score de 0,703 signifie concrètement que 30 % des prédictions sont incorrectes. Ces erreurs se déclinent en deux typologies aux conséquences commerciales opposées. Les faux positifs (prédire un achat alors que l'utilisateur abandonne) risquent de générer des notifications inutiles qui nuisent à l'expérience utilisateur sans apporter de conversion. Les faux négatifs (prédire un abandon alors que l'utilisateur allait acheter de lui-même) sont potentiellement plus coûteux : un coupon de réduction de 20 % envoyé automatiquement à un client qui aurait acheté plein tarif engendre une érosion de marge directe sur une vente qui se serait réalisée sans intervention.

12.4.2 Le Principe de Supervision Humaine

Pour contenir ces risques, le fichier `consomateur.py` applique des seuils de confiance différenciés avant tout déclenchement d'action. Une notification légère peut être générée pour des scores d'intention supérieurs à 0,75, tandis que les actions à impact financier requièrent une probabilité supérieure à 0,85. Les recommandations suivent des règles métier déterministes codées explicitement dans le pipeline Spark (coupon pour les abandons, livraison gratuite pour les ajouts au panier) plutôt que de résulter d'une décision autonome du modèle. Le dashboard Streamlit du Chapitre 9 expose en temps réel les anomalies détectées aux opérateurs humains, qui valident l'action corrective

avant tout envoi effectif. Le modèle produit des suggestions, la décision finale reste humaine.

12.4.3 Limites du Projet

Trois limitations importantes méritent d'être documentées honnêtement. Les données d'entraînement sont entièrement synthétiques, avec une distribution des actions quasi-uniforme qui ne ressemble pas à du trafic e-commerce réel, et les métriques F1 obtenues seraient vraisemblablement différentes sur de vraies données de production. Aucun mécanisme de détection de dérive temporelle n'est implémenté : si les comportements d'achat évoluent, le modèle se dégradera progressivement sans que le système le signale, ce qui nécessiterait en production un pipeline de réentraînement périodique. Enfin, l'explicabilité des prédictions par SHAP n'a pas été implémentée dans le périmètre de ce projet et constitue un axe d'amélioration prioritaire identifié au Chapitre 13.

Chapitre 13

Conclusion et Perspectives

13.1 Bilan Technique du Projet

Ce projet de Big Data nous a permis de concevoir et implémenter une architecture complète de traitement de données en temps réel pour un système e-commerce. L'ensemble de la chaîne de traitement a été réalisé avec succès, depuis la génération des données jusqu'au dashboard interactif en passant par le stockage NoSQL et la modélisation par apprentissage automatique.

Apache Kafka a joué le rôle central de bus de messagerie distribué, orchestrant trois flux de données distincts (clickstream, transactions et service client) avec une latence inférieure à la seconde. Le producteur `producteur.py` a généré en continu des événements réalistes en s'appuyant sur les APIs FakeStore et DummyJSON, simulant fidèlement le comportement d'utilisateurs réels sur une plateforme e-commerce.

Apache Spark Structured Streaming a assuré le traitement distribué de ces flux en temps réel, avec des micro-batches de 10 secondes. Le pipeline de traitement a intégré le nettoyage des données, le calcul du score d'engagement, l'enrichissement par recommandations automatiques, et l'intégration du pipeline NLP complet basé sur TextBlob et NLTK.

Le stockage NoSQL a été assuré par MongoDB comme base principale, avec une comparaison approfondie avec Cassandra. Les données enrichies ont été persistées dans trois collections distinctes, permettant des analyses ultérieures via des agrégations MongoDB et des requêtes Spark.

La partie modélisation a comparé trois algorithmes de classification (Random Forest, Gradient Boosting (XGBoost) et Régression Logistique) pour prédire l'intention d'achat des utilisateurs. Le meilleur modèle a été sauvegardé sous forme de fichier `.pkl` et intégré dans le pipeline de streaming pour des prédictions en temps réel.

Enfin, le dashboard interactif développé avec Streamlit offre une visualisation en temps réel de l'ensemble des métriques clés du système, avec un rafraîchissement automatique toutes les 10 secondes.

13.2 Limites et Difficultés Rencontrées

La réalisation de ce projet n'a pas été sans difficultés. Plusieurs obstacles techniques ont été rencontrés et surmontés tout au long du développement.

La première difficulté majeure a concerné l'environnement d'exécution. L'utilisation d'une machine virtuelle VirtualBox sous Ubuntu a engendré des contraintes importantes en termes de ressources mémoire et de processeur. Les services Docker (Kafka, MongoDB, Cassandra et Spark) consomment ensemble une quantité considérable de RAM, ce qui a nécessité une gestion rigoureuse des ressources et des ajustements constants des paramètres de configuration Spark tels que `spark.executor.memory` et `spark.driver.memory`.

La connectivité réseau a représenté un second défi de taille. La machine virtuelle n'avait pas toujours accès à internet, ce qui a compliqué le téléchargement des images Docker et l'accès aux APIs externes utilisées par le producteur. Des solutions alternatives ont dû être développées pour contourner ces limitations, notamment la configuration des serveurs DNS et des miroirs Docker.

La compatibilité des versions entre Spark, Scala, et le connecteur Kafka a également posé des problèmes. Les versions `spark-sql-kafka-0-10_2.12` et `spark-sql-kafka-0-10_2.13` ne sont pas interchangeables, et des erreurs de résolution de dépendances ont nécessité plusieurs itérations avant de trouver la combinaison compatible avec notre version de Spark installée dans `/opt/spark`.

La gestion de l'authentification MongoDB a été une source de confusion. Le mécanisme `MONGO_INITDB_ROOT_USERNAME` du conteneur Docker n'a pas fonctionné comme attendu, obligeant à opter pour une configuration sans authentification pour garantir le bon fonctionnement du pipeline de streaming.

Enfin, le déséquilibre des classes dans les données d'entraînement, les abandons étant moins fréquents que les autres actions, a nécessité l'utilisation de la technique SMOTE pour rééquilibrer le dataset avant l'entraînement des modèles de machine learning.

13.3 Perspectives et Améliorations

Ce projet constitue une base solide qui peut être considérablement enrichie selon plusieurs axes d'amélioration.

Sur le plan de l'infrastructure, le déploiement sur un cluster cloud tel qu'AWS, Google Cloud Platform ou Microsoft Azure permettrait de lever les contraintes de ressources rencontrées en environnement local. L'utilisation de Kubernetes pour orchestrer les conteneurs Docker offrirait une scalabilité automatique en fonction de la charge, garantissant la haute disponibilité du système même en cas de pic de trafic.

Concernant le pipeline de données, l'intégration de Apache Flink à la place ou en complément de Spark Streaming permettrait de réduire la latence de traitement à quelques millisecondes, contre 10 secondes actuellement. L'ajout d'un système de gestion de schémas via le Schema Registry de Confluent renforcerait la robustesse du pipeline face aux évolutions des structures de données.

Du côté de l'intelligence artificielle, plusieurs améliorations significatives sont envisageables. L'implémentation de modèles de Deep Learning tels que LSTM ou Transformer pour l'analyse de sentiment remplacerait avantageusement TextBlob, offrant une précision nettement supérieure. Un système de recommandation collaboratif basé sur les historiques d'achat enrichirait les recommandations actuellement basées sur des règles simples. L'intégration de MLflow permettrait de gérer le cycle de vie des modèles avec un suivi automatique des expériences, de la validation et du déploiement.

Pour le dashboard, l'ajout d'alertes en temps réel par email ou SMS lorsque certains seuils sont dépassés (taux d'abandon anormal, pic de tickets support, baisse du revenu) apporterait une valeur opérationnelle immédiate. L'intégration d'un système de A/B testing permettrait de mesurer l'impact réel des recommandations générées sur le comportement des utilisateurs.

Enfin, concernant la qualité des données, la mise en place d'un pipeline de validation automatique des schémas à l'entrée de Kafka, couplé à un système de monitoring avec Prometheus et Grafana, permettrait de détecter et corriger les anomalies de données en temps réel.

13.4 Apports Personnels et Compétences Acquis

Ce projet nous a apporté une expérience concrète et approfondie dans de nombreux domaines du Big Data et de l'ingénierie des données, compétences particulièrement recherchées dans l'industrie aujourd'hui.

Sur le plan technique, nous avons acquis une maîtrise pratique des technologies de streaming distribué, notamment Apache Kafka et Apache Spark Structured Streaming. La mise en place d'une architecture de bout en bout, de la production de données jusqu'au dashboard en passant par le traitement et le stockage, nous a confrontés aux défis réels des systèmes distribués : gestion de la latence, tolérance aux pannes,

checkpointing, et cohérence des données.

La manipulation de bases de données NoSQL de nature différente (MongoDB orienté documents et Cassandra orienté colonnes) nous a permis de comprendre concrètement les compromis CAP (Consistency, Availability, Partition tolerance) et de choisir la technologie adaptée selon les besoins fonctionnels du projet. Cette expérience comparative est précieuse pour des choix d'architecture futurs.

L'intégration du traitement du langage naturel dans un pipeline de streaming temps réel a été une expérience enrichissante. L'utilisation combinée de TextBlob pour l'analyse de sentiment et de NLTK pour la tokenisation et la lemmatisation nous a permis d'extraire de la valeur des données textuelles non structurées, une compétence fondamentale en data science moderne.

La partie modélisation machine learning nous a appris à gérer un pipeline ML complet en production : préparation des données, gestion du déséquilibre des classes avec SMOTE, comparaison rigoureuse de plusieurs algorithmes avec des métriques multiples (accuracy, précision, rappel, F1-score et AUC-ROC) et sauvegarde des modèles pour déploiement en temps réel.

Sur le plan de la gestion de projet, nous avons développé une approche méthodique face aux problèmes complexes : décomposition en sous-problèmes, débogage itératif, et adaptation continue face aux contraintes techniques imprévues. La capacité à trouver des solutions alternatives, comme la configuration sans authentification MongoDB ou l'utilisation de données locales en l'absence d'internet, témoigne d'une résilience technique essentielle en ingénierie.

Enfin, ce projet nous a sensibilisés à l'importance de la documentation technique, de la reproductibilité des expériences, et de la communication des résultats via des visualisations claires et interactives. Ces compétences transversales constituent des atouts majeurs pour notre future carrière dans le domaine du Big Data et de l'intelligence artificielle.

Références

1. Apache Kafka Documentation : <https://kafka.apache.org/documentation/>
2. Apache Spark Structured Streaming : <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
3. MongoDB Manual : <https://www.mongodb.com/docs/manual/>
4. Streamlit Documentation : <https://docs.streamlit.io/>
5. Scikit-learn : <https://scikit-learn.org/stable/>